

MULESOFT – AWS INTEGRATION PROBLEM SET ·
VOLUME 1 OF 5

100 MuleSoft–AWS Integration Problems

Volume 1 — S3 Fundamentals. 20 fully worked, real-project problems: uploading, downloading, listing, copying, archiving, and securing objects in Amazon S3 from Mule, plus the practical file-handling patterns a MuleSoft developer actually needs on a live integration.

Problems 1–20

Anypoint Studio

Postman

MuleSoft Developer Edition

How This Book Works

This series is written for MuleSoft developers building real AWS integrations — not AWS solutions architects or Lambda-first developers. Every problem stays entirely on the MuleSoft side: Anypoint Connector for Amazon S3 (and, across the series, SQS, SNS, Lambda, DynamoDB, and the rest of the AWS connector family), DataWeave, error handling, and Anypoint Studio. Nothing in this book requires writing custom AWS SDK code, Lambda business logic, or infrastructure-as-code — every solution is a MuleSoft flow that talks to AWS.

Every problem in this book follows the same ten-part structure:

- 1 **Problem Statement** — the real-project scenario in one or two sentences.
- 2 **Detailed Explanation** — why this comes up on a real integration and what's actually at stake.
- 3 **Connectors & Components Used** — the exact Mule building blocks this problem touches.
- 4 **Steps to Create the Mule Flow** — the conceptual plan, before any tool is opened.
- 5 **Anypoint Studio Canvas Flow Structure** — how the flow looks laid out on the canvas.
- 6 **Mule Configuration XML** — production-quality flow XML in a syntax-highlighted code editor window.
- 7 **Explanation of the Code** — a line-by-line walkthrough of the parts that matter.
- 8 **Steps to Build It in Anypoint Studio** — the literal clicks: palette, canvas, properties panel.
- 9 **Testing in Postman** — a real request, with sample input and sample response.
- 10 **Briefing All the Steps in Words** — the whole problem recapped in a short paragraph.

Volume 1 — S3 Fundamentals

Every MuleSoft–AWS integration, no matter how sophisticated it eventually becomes, starts with moving files reliably in and out of Amazon S3. This volume builds those fundamentals the way a real project actually needs them — with proper error handling, streaming for large files, security, and the polling and tagging patterns that hold up past a demo.

#1 Uploading a File Received via HTTP Listener to an S3 Bucket

AWS

PROBLEM STATEMENT

A partner system uploads invoice PDFs to a MuleSoft-exposed API endpoint. Every file needs to land in a specific S3 bucket, under a key that includes the date, without the partner ever knowing an S3 bucket is involved.

DETAILED EXPLANATION

This is the entry point for almost every MuleSoft–AWS file integration: something outside AWS needs to put a file into S3, and it shouldn't need AWS credentials, an SDK, or any knowledge of bucket naming to do it. Putting a Mule API in front of S3 means the partner only ever talks to one stable HTTP contract, while Mule owns the translation into an S3 PutObject call — including deciding the key structure, which matters enormously once a bucket holds millions of objects and someone needs to find today's invoices without listing the whole bucket.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

AWS Amazon S3 Connector – Upload

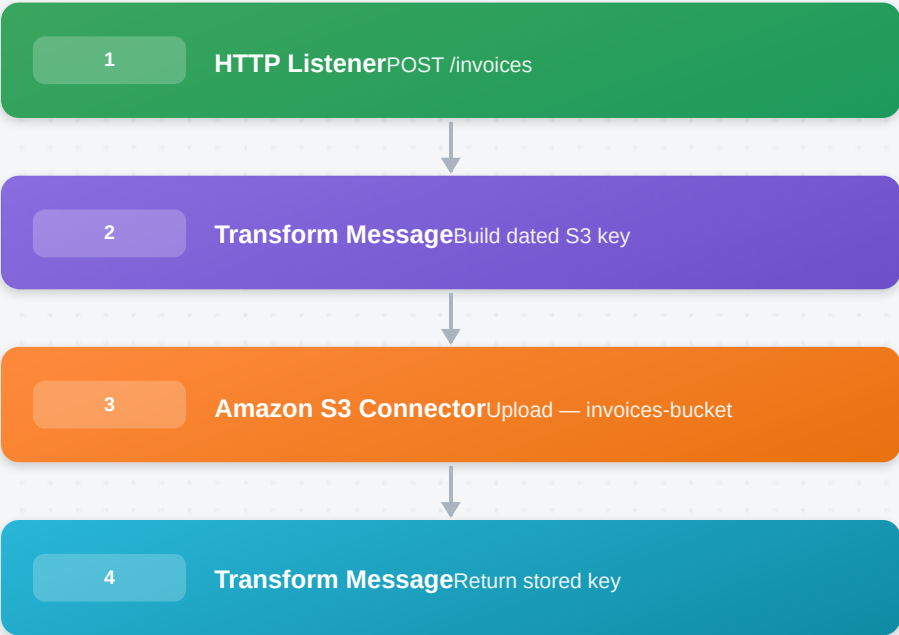
DW Transform Message

ERR On Error Propagate

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP POST endpoint that accepts a multipart file upload.
- 2 Build a date-partitioned S3 key from the current timestamp and the original filename.
- 3 Call the S3 Connector's Upload operation with the file bytes, bucket name, and computed key.
- 4 Return the final S3 key to the caller as confirmation.

UPLOAD-INVOICE-FLOW



MULE CONFIGURATION XML

```

<flow name="upload-invoice-flow">
  <http:listener path="/invoices" config-
ref="HTTP_Listener_config"/>
  <set-variable variableName="s3Key" value="#['invoices/' ++ (now()
as String {format: 'yyyy/MM/dd'}) ++ '/' ++
attributes.parts.file.filename]"/>
  <s3:upload config-ref="S3_Config" doc:name="Upload to S3"
    bucketName="invoices-bucket" key="#[vars.s3Key]">
    <s3:content>#[payload.parts.file.content]</s3:content>
  </s3:upload>
  <ee:transform doc:name="Build response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  bucket: 'invoices-bucket',
  key: vars.s3Key,
  status: 'UPLOADED'
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

set-variable building s3Key before the upload

Computing the key once into a variable, rather than inline inside the Upload operation, means the same value can be reused in the response transform without recalculating `now()` a second time and risking a different timestamp.

s3:content bound to payload.parts.file.content

A multipart HTTP request exposes each part's binary content under `parts`. `<name>.content`, so the S3

connector streams that content directly into the PutObject call without Mule ever buffering the whole file in memory.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Create a new Mule Project, then drag an **HTTP Listener** from the Mule Palette onto the canvas.
- 2 Double-click the listener, set **Path** to **/invoices**, and create a new **HTTP_Listener_config** on port 8081.
- 3 Drag a **Set Variable** component, name it **s3Key**, and write the dated-key expression in its **Value** field.
- 4 Search the Mule Palette for **Amazon S3**, drag the **Upload** operation onto the canvas, and set **Bucket Name** and **Key**.
- 5 Click the red flag icon next to the S3 config, choose **Basic Authentication**, and enter the AWS Access Key and Secret Key (or an assumed IAM role).
- 6 Drag a **Transform Message** after the Upload operation to build the JSON confirmation, then save and run the project.

TESTING IN POSTMAN

POST

https://localhost:8081/invoices

Header

Value

Content-Type

multipart/form-data

SAMPLE REQUEST (POSTMAN BODY → FORM-DATA)

FORM-DATA

Key: file Type: File Value: invoice-4471.pdf

SAMPLE RESPONSE

JSON

```
{
  "bucket": "invoices-bucket",
  "key": "invoices/2026/08/28/invoice-4471.pdf",
  "status": "UPLOADED"
}
```

BRIEFING ALL THE STEPS IN WORDS

An HTTP Listener accepts the invoice as a multipart upload, a Set Variable step computes a date-partitioned S3 key from the filename and current date, the S3 Connector's Upload operation streams the file straight into the bucket under that key, and a final transform hands the caller back the exact location the file now lives — a four-step round trip that keeps every future partner from needing to know S3 exists at all.

#2

Downloading a File from S3 and Returning It as an HTTP Response

AWS

PROBLEM STATEMENT

A customer portal needs to let users download a generated report by ID, but the reports live in a private S3 bucket the browser can never reach directly.

DETAILED EXPLANATION

Keeping the bucket private and proxying the download through Mule is the safer default for anything that isn't meant to be publicly cacheable: access control, auditing, and even on-the-fly transformation of the file all stay inside the integration layer instead of depending on bucket policies the browser could otherwise bypass. The tradeoff is that Mule now sits in the data path for every byte of every download, so streaming the S3 object straight through to the HTTP response — instead of buffering it fully in memory first — is what keeps this pattern viable for anything larger than a tiny file.

CONNECTORS & COMPONENTS USED

CORE

HTTP Listener

AWS

Amazon S3 Connector – Get Object

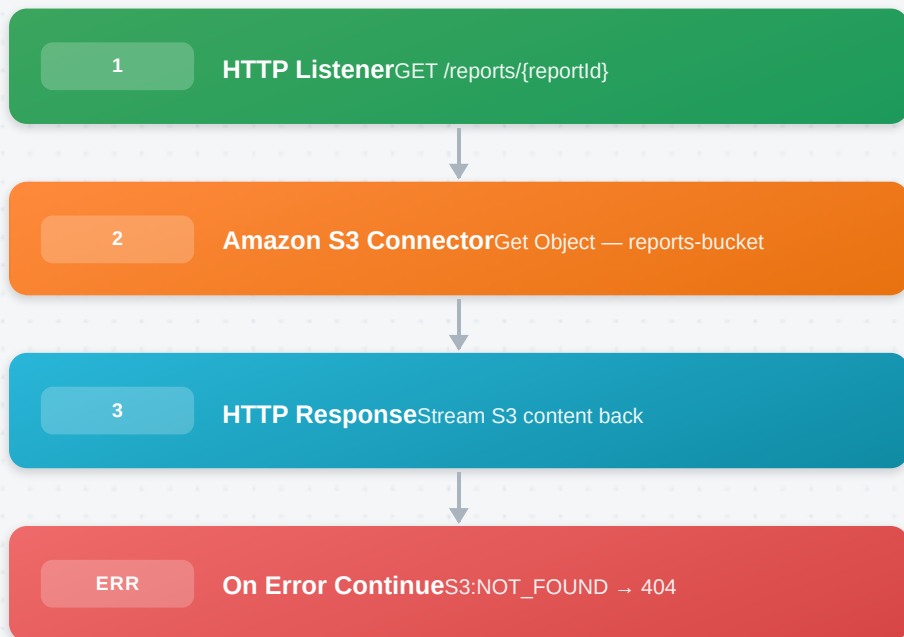
ERR

On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP GET endpoint that takes a report ID as a URI parameter.
- 2 Resolve the report ID to its S3 key (here, a direct mapping to keep the example focused).
- 3 Call the S3 Connector's Get Object operation and stream the result back as the HTTP response.
- 4 Handle the not-found case with a clean 404 instead of a raw AWS error.

DOWNLOAD-REPORT-FLOW



MULE CONFIGURATION XML

```

<flow name="download-report-flow">
  <http:listener path="/reports/{reportId}" config-
ref="HTTP_Listener_config"/>
  <s3:get-object config-ref="S3_Config" doc:name="Get report from
S3"
    bucketName="reports-bucket"
    key="#['reports/' ++ attributes.uriParams.reportId ++
'.pdf']"/>
  <set-payload value="#[payload]" mimeType="application/pdf"/>
  <error-handler>
    <on-error-continue type="S3:NOT_FOUND">
      <set-variable variableName="httpStatus" value="404"/>
      <ee:transform>
        <ee:message>
          <ee:set-payload>#[%dw 2.0
output application/json
---
{ error: 'REPORT_NOT_FOUND' }]</ee:set-payload>
        </ee:message>
      </ee:transform>
    </on-error-continue>
  </error-handler>
</flow>

```

EXPLANATION OF THE CODE

set-payload mimeType="application/pdf"

Explicitly tagging the S3 object's binary stream with its real content type is what makes the HTTP response come back with the correct **Content-Type** header, so the browser (or Postman) renders or downloads it correctly instead of treating it as generic bytes.

on-error-continue type="S3:NOT_FOUND"

Scoping the handler to the connector's specific **NOT_FOUND** error type means only a missing object gets converted to a clean 404; any other S3

failure (a permissions problem, a network error) still propagates as a real error instead of being silently mapped to "not found".

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/reports/{reportId}`.
- 2 Drag the S3 **Get Object** operation, set **Bucket Name** to `reports-bucket`, and write the key expression from the URI parameter.
- 3 Add a **Set Payload** component with **MIME Type** set to `application/pdf` so the response headers are correct.
- 4 Right-click the flow and add an **Error Handler**, then drag **On Error Continue** and set its **Type** to `S3:NOT_FOUND`.
- 5 Inside the error handler, set **httpStatus** to `404` and build a JSON error body, then save and run.

TESTING IN POSTMAN

GET

`https://localhost:8081/reports/RPT-8821`

SAMPLE RESPONSE

RESPONSE

```
Content-Type: application/pdf
Status: 200 OK
<binary PDF content streamed to Postman's Save Response → Save to a
file>
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a report ID in the URL, the S3 Connector's Get Object operation retrieves the matching PDF from a private bucket, and a Set Payload step tags it with the right content type so it streams back to the caller looking exactly like a native file download — with a dedicated error branch turning a missing object into a clean 404 instead of a raw AWS exception.

#3 Listing All Objects in an S3 Bucket with a Given Prefix

AWS

PROBLEM STATEMENT

An operations dashboard needs to show every invoice file uploaded for a given date, without downloading any of the actual file contents.

DETAILED EXPLANATION

S3 has no real folder structure — what looks like a folder is just a shared key prefix — so "show me today's invoices" really means "list every object whose key starts with `invoices/2026/08/28/`". This is a completely different, and much cheaper, operation than retrieving objects: it returns keys, sizes, and last-modified timestamps in one paginated call, which is exactly what a dashboard needs to render a file list without pulling a single byte of actual content across the wire.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

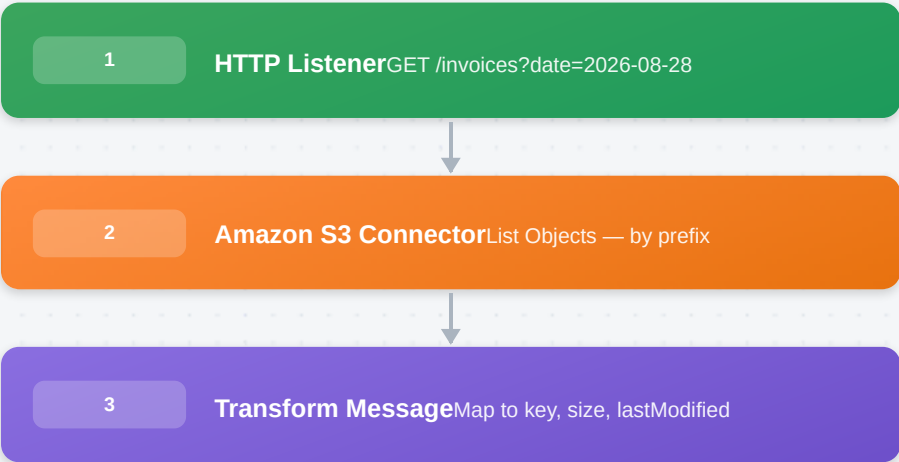
AWS Amazon S3 Connector – List Objects

DW Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP GET endpoint that accepts a date as a query parameter.
- 2 Build the matching key prefix from that date.
- 3 Call the S3 Connector's List Objects operation, scoped to that prefix.
- 4 Transform the raw object summaries into a compact JSON array for the dashboard.

LIST-INVOICES-FLOW



MULE CONFIGURATION XML

```

<flow name="list-invoices-flow">
  <http:listener path="/invoices" config-
ref="HTTP_Listener_config"/>
  <s3:list-objects config-ref="S3_Config" doc:name="List invoice
objects"
    bucketName="invoices-bucket"
    prefix="#['invoices/' ++ (attributes.queryParams.date replace
'-' with '/') ++ '/']"/>
  <ee:transform doc:name="Shape file list">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
payload map {
  key: $.key,
  sizeBytes: $.size,
  lastModified: $.lastModified
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

`prefix built by replacing '-' with '/' in the date`

Because the upload flow in Problem #1 partitions keys as yyyy/MM/dd, converting an incoming 2026-08-28 query parameter into 2026/08/28/ is what makes the prefix line up with how objects were actually stored.

`payload map { key, sizeBytes, lastModified }`

The List Objects operation returns a fairly verbose object summary per item (including storage class and owner); mapping down to just three fields keeps

the dashboard payload small and focused on what it actually renders.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to **/invoices**.
- 2 Drag the S3 **List Objects** operation, set **Bucket Name**, and write the prefix expression in the **Prefix** field.
- 3 Drag a **Transform Message** component and write the DataWeave **map** to shape the response array.
- 4 Save and run, then test with a date that has uploaded files and one that doesn't (expecting an empty array).

TESTING IN POSTMAN

GET

https://localhost:8081/invoices?date=2026-08-28

SAMPLE RESPONSE

JSON

```
[
  { "key": "invoices/2026/08/28/invoice-4471.pdf", "sizeBytes": 88213,
    "lastModified": "2026-08-28T14:02:11Z" },
  { "key": "invoices/2026/08/28/invoice-4472.pdf", "sizeBytes": 71040,
    "lastModified": "2026-08-28T14:07:52Z" }
]
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a date, that date is converted into the matching key prefix, the S3 Connector's List Objects operation returns every matching object's metadata without touching file contents, and a transform trims that down to exactly the three fields the dashboard needs — a fast, content-free way to answer "what's in this bucket".

#4 Copying an Object Between Two S3 Buckets

AWS

PROBLEM STATEMENT

A file lands in a shared "incoming" bucket used by multiple partners, and once validated it needs to be duplicated into a department-specific bucket for downstream processing — without ever leaving Mule's network path or being re-uploaded from scratch.

DETAILED EXPLANATION

The naive way to move data between buckets — download the object, then upload it again — pulls the entire file through Mule twice for no reason. S3's server-side Copy Object operation instead tells AWS to duplicate the object directly, bucket to bucket, entirely within AWS's own infrastructure; Mule just issues the instruction. For large files or high volumes, this is the difference between an integration that scales and one that saturates its own network egress moving bytes it never needed to see.

CONNECTORS & COMPONENTS USED

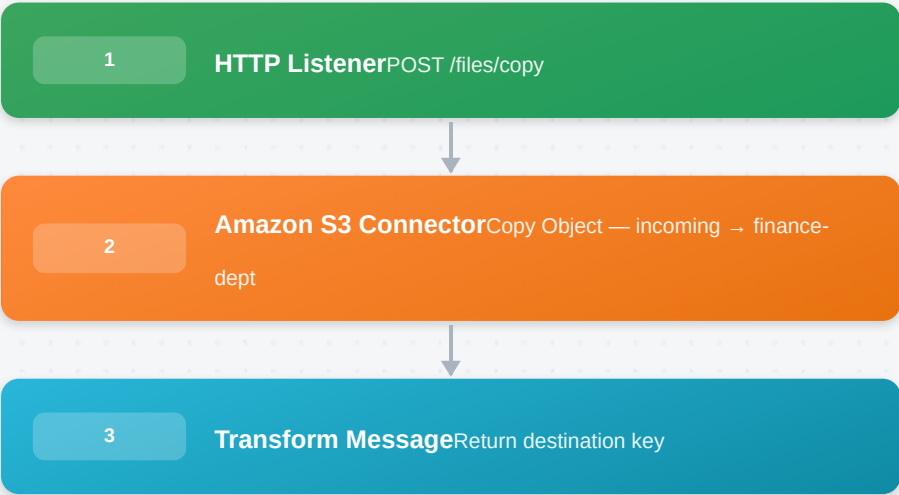
CORE HTTP Listener

AWS Amazon S3 Connector – Copy Object

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP POST endpoint that accepts a source key.
- 2 Call the S3 Connector's Copy Object operation with the source bucket/key and destination bucket/key.
- 3 Confirm the copy succeeded and return the destination location.

COPY-TO-DEPARTMENT-FLOW



MULE CONFIGURATION XML

```

<flow name="copy-to-department-flow">
  <http:listener path="/files/copy" config-
ref="HTTP_Listener_config"/>
  <s3:copy-object config-ref="S3_Config" doc:name="Copy to finance
dept bucket"
    sourceBucketName="incoming-bucket"
    sourceKey="#[payload.sourceKey]"
    destinationBucketName="finance-dept-bucket"
    destinationKey="#[payload.sourceKey]"/>
  <ee:transform doc:name="Build response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  destinationBucket: 'finance-dept-bucket',
  destinationKey: payload.sourceKey,
  status: 'COPIED'
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

`sourceBucketName / destinationBucketName as separate attributes`

Copy Object is the one S3 operation that spans two buckets at once, so the connector exposes both a source and destination bucket/key pair rather than the usual single

bucketName/key
combination.

`destinationKey` reusing `payload.sourceKey` unchanged

Keeping the key identical across buckets is a deliberate choice here — it means any downstream tooling that already knows the key from the incoming bucket can find the same file in the department bucket without a lookup.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/files/copy`.
- 2 Drag the S3 `Copy Object` operation and fill in `Source Bucket Name`, `Source Key`, `Destination Bucket Name`, and `Destination Key`.
- 3 Drag a `Transform Message` after it to confirm the copy in the response, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/files/copy

Header	Value
Content-Type	application/json

SAMPLE REQUEST BODY



JSON

```
{ "sourceKey": "invoices/2026/08/28/invoice-4471.pdf" }
```

SAMPLE RESPONSE



JSON

```
{  
  "destinationBucket": "finance-dept-bucket",  
  "destinationKey": "invoices/2026/08/28/invoice-4471.pdf",  
  "status": "COPIED"  
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a source key, the S3 Connector's Copy Object operation instructs AWS to duplicate that object directly from the incoming bucket into the finance department's bucket without the bytes ever passing through Mule, and a transform confirms the new location back to the caller — a server-side copy that stays fast and cheap no matter how large the file is.

#5 Generating a Pre-Signed URL for Temporary S3 Object Access

AWS

PROBLEM STATEMENT

A customer-facing web app wants to let users download their own generated statements directly from S3 — without routing every download through Mule, and without making the bucket public.

DETAILED EXPLANATION

Proxying every download through Mule (Problem #2's pattern) works, but it isn't free — every byte transferred is a byte Mule's infrastructure has to move. A pre-signed URL sidesteps that entirely: Mule uses its AWS credentials to generate a URL that embeds a cryptographic signature and an expiry time, and AWS itself validates that signature when the browser requests it directly. The bucket stays fully private, but for a short window, that one specific URL grants access to that one specific object — the standard AWS pattern for temporary, self-expiring access without proxying traffic.

CONNECTORS & COMPONENTS USED

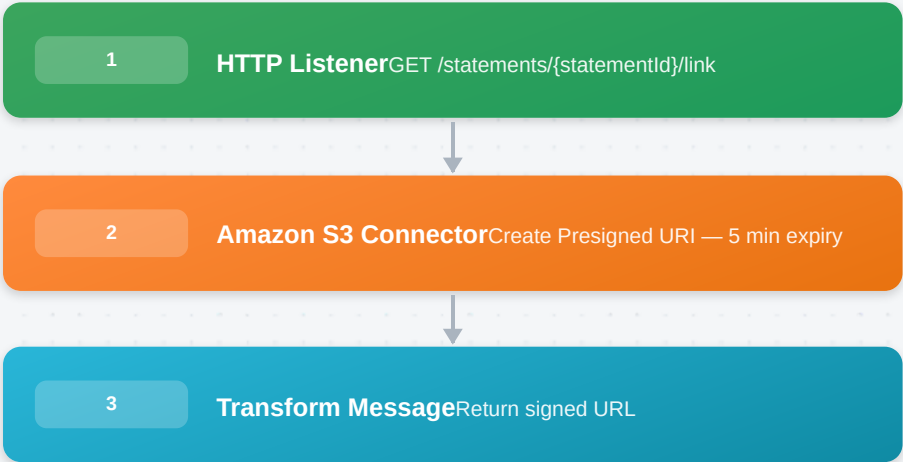
CORE HTTP Listener

AWS Amazon S3 Connector – Create Presigned URI

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP GET endpoint that takes a statement ID.
- 2 Resolve the statement ID to its S3 key.
- 3 Call the S3 Connector's Create Presigned URI operation with a short expiry.
- 4 Return the signed URL to the web app instead of the file itself.

PRESIGN-STATEMENT-FLOW



MULE CONFIGURATION XML

```

<flow name="presign-statement-flow">
  <http:listener path="/statements/{statementId}/link" config-
ref="HTTP_Listener_config"/>
  <s3:create-presigned-uri config-ref="S3_Config" doc:name="Presign
statement URL"
    bucketName="statements-bucket"
    key="#['statements/' ++ attributes.uriParams.statementId ++
'.pdf']"
    method="GET"
    expiration="5"
    timeUnit="MINUTES"/>
  <ee:transform doc:name="Build response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  downloadUrl: payload,
  expiresInMinutes: 5
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

expiration="5" timeUnit="MINUTES"

Keeping the expiry short and specific to the use case — a user downloading their own statement right now — limits the damage if the URL is ever accidentally shared or logged somewhere it shouldn't be, since it simply stops working five minutes after issue.

method="GET" matching the operation the caller will perform

A presigned URL's signature is bound to the HTTP method it was generated for;

requesting GET here means the resulting URL only works for downloading the object, not for overwriting or deleting it.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/statements/{statementId}/link`.
- 2 Drag the S3 **Create Presigned URI** operation, set **Bucket Name** and the **Key** expression.
- 3 Set **Method** to **GET**, **Expiration** to **5**, and **Time Unit** to **MINUTES**.
- 4 Drag a **Transform Message** to wrap the returned URL in a small JSON body, then save and run.

TESTING IN POSTMAN

GET

`https://localhost:8081/statements/STMT-2026-08/link`

SAMPLE RESPONSE

JSON

```
{
  "downloadUrl": "https://statements-
bucket.s3.amazonaws.com/statements/STMT-2026-08.pdf?X-Amz-
Signature=...&X-Amz-Expires=300",
  "expiresInMinutes": 5
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a statement ID, the S3 Connector's Create Presigned URI operation generates a signed, time-limited GET URL for the matching object, and a transform hands that URL

back to the web app — letting the browser download the file directly from AWS while the bucket itself stays completely private and Mule never touches the file's bytes.

#6 Deleting an Object from S3 After Processing

AWS

PROBLEM STATEMENT

Once an uploaded invoice has been successfully parsed and loaded into the finance system, the raw file in the "pending" bucket needs to be removed so it isn't picked up and processed twice.

DETAILED EXPLANATION

Deletion is simple mechanically, but the sequencing is what actually matters on a real project: the object should only be deleted after downstream processing has genuinely succeeded, never before or in parallel with it, or a transient failure downstream ends up destroying the only copy of the source data. Placing the Delete Object call at the very end of a successful flow — after the finance system has acknowledged the load — is what makes deletion safe instead of a data-loss risk.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

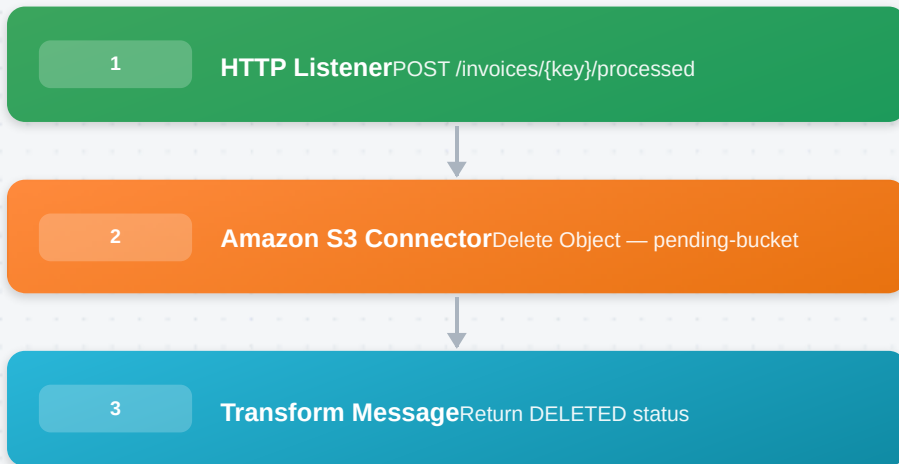
AWS Amazon S3 Connector – Delete Object

ERR On Error Propagate

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP POST endpoint that confirms an invoice was successfully processed.
- 2 Call the S3 Connector's Delete Object operation for that invoice's key.
- 3 Return a confirmation once the delete succeeds; propagate any failure without silently swallowing it.

DELETE-PROCESSED-INVOICE-FLOW



MULE CONFIGURATION XML

```
<flow name="delete-processed-invoice-flow">
  <http:listener path="/invoices/{key}/processed" config-
ref="HTTP_Listener_config"/>
  <s3:delete-object config-ref="S3_Config" doc:name="Delete
processed invoice"
    bucketName="pending-invoices-bucket"
    key="#[attributes.uriParams.key]"/>
  <ee:transform doc:name="Build response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{ key: attributes.uriParams.key, status: 'DELETED' }]]</ee:set-
payload>
    </ee:message>
  </ee:transform>
</flow>
```

EXPLANATION OF THE CODE

no error-handler overriding the default propagate behavior

Deliberately leaving Mule's default error handling in place means a Delete Object failure — a permissions issue, a network blip — surfaces as a real error to the caller rather than being masked, which matters because a caller that thinks deletion succeeded when it didn't will never retry it.

key taken directly from the URI path parameter

Accepting the exact key that was returned by the original upload (Problem #1) rather than reconstructing it avoids any risk of the delete flow computing a slightly different key and deleting nothing, or the wrong object.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/invoices/{key}/processed`.
- 2 Drag the S3 **Delete Object** operation, set **Bucket Name**, and bind **Key** to the URI parameter.
- 3 Drag a **Transform Message** to confirm deletion, then save and run — leave error handling on its default so failures still surface.

TESTING IN POSTMAN

POST

https://localhost:8081/invoices/invoices%2F2026%2F08%2F28%2Finvoice-4471.pdf/processed

SAMPLE RESPONSE

JSON

```
{
  "key": "invoices/2026/08/28/invoice-4471.pdf",
  "status": "DELETED"
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts confirmation that an invoice has been fully processed downstream, the S3 Connector's Delete Object operation removes exactly that key from the pending bucket, and a transform confirms the deletion — a deliberately last step in the pipeline, since removing the source file is only ever safe once everything that needed it has already succeeded.

#7 Reading S3 Object Metadata Before Processing a File

AWS

PROBLEM STATEMENT

Before downloading and parsing an uploaded file, a flow needs to check its size and content type — rejecting anything over 50 MB or not a PDF — without pulling the actual file content across the network first.

DETAILED EXPLANATION

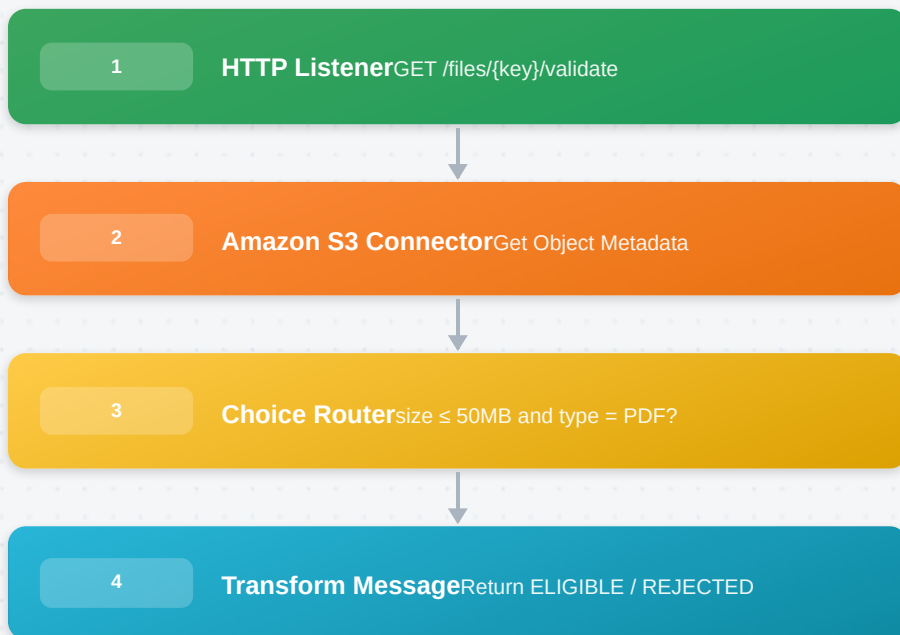
S3 stores metadata — size, content type, last-modified time, and any custom metadata set at upload — separately from the object body, and a HEAD-style request can retrieve just that metadata without transferring the body at all. Checking size and type this way before committing to a full download is what keeps a misbehaving upstream system (or a malicious one) from being able to force Mule to pull a multi-gigabyte file into memory just to discover it should have been rejected.

CONNECTORS & COMPONENTS USED

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP GET endpoint that validates a given key.
- 2 Call the S3 Connector's Get Object Metadata operation for that key.
- 3 Branch on size and content type to decide whether the file is eligible for processing.
- 4 Return a clear eligibility verdict, without ever calling Get Object.

VALIDATE-FILE-METADATA-FLOW



MULE CONFIGURATION XML

```

<flow name="validate-file-metadata-flow">
  <http:listener path="/files/{key}/validate" config-
ref="HTTP_Listener_config"/>
  <s3:get-object-metadata config-ref="S3_Config" doc:name="Get file
metadata"
    bucketName="pending-invoices-bucket"
    key="#[attributes.uriParams.key]" target="fileMeta"/>
  <choice>
    <when expression="#[vars.fileMeta.contentLength <= 52428800 and
vars.fileMeta.contentType == 'application/pdf']">
      <ee:transform>
        <ee:message><ee:set-payload>#[%dw 2.0 output
application/json --- { eligible: true, reason: 'OK' }]</ee:set-
payload></ee:message>
      </ee:transform>
    </when>
    <otherwise>
      <ee:transform>
        <ee:message><ee:set-payload>#[%dw 2.0 output
application/json --- { eligible: false, reason:
'SIZE_OR_TYPE_NOT_SUPPORTED' }]</ee:set-payload></ee:message>
      </ee:transform>
    </otherwise>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

target="fileMeta" on Get Object Metadata

Writing the metadata result into a named variable rather than the payload keeps the original request context available and makes the Choice router's expression read cleanly as `vars.fileMeta....`

52428800 as the byte-count limit

50 MB expressed in bytes ($50 \times 1024 \times 1024$) is what the connector's `contentLength` field is measured in, so

the comparison has to be done in bytes rather than the more human-readable "50MB".

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/files/{key}/validate`.
- 2 Drag the S3 **Get Object Metadata** operation, set **Bucket Name** and **Key**, and set **Target Variable** to `fileMeta`.
- 3 Drag a **Choice** router; in its **when** branch write the size-and-type expression.
- 4 Add matching **Transform Message** components inside each branch, then save and run.

TESTING IN POSTMAN

GET

`https://localhost:8081/files/invoices%2F2026%2F08%2F28%2Finvoice-4471.pdf/validate`

SAMPLE RESPONSE

JSON

```
{ "eligible": true, "reason": "OK" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a key, the S3 Connector's Get Object Metadata operation retrieves size and content type without fetching the file body, and a Choice router turns those two values into a clean eligible/rejected verdict — keeping a full download off the table until a file has already proven it's worth processing.

#8 Uploading a Large File to S3 Using Multipart Upload

AWS

PROBLEM STATEMENT

A nightly data export from a legacy system produces a single 2 GB file that needs to reach S3 reliably, without a single network blip forcing the entire transfer to restart from zero.

DETAILED EXPLANATION

A single PutObject call for a multi-gigabyte file is fragile: if the connection drops at 95% complete, the whole upload has to start over. S3's multipart upload API splits the object into independently-uploaded parts (5 MB minimum each, except the last), lets any individual part be retried without touching the others, and only assembles the final object once every part is confirmed — the standard approach for anything large enough that a full restart would actually hurt.

CONNECTORS & COMPONENTS USED

CORE Scheduler

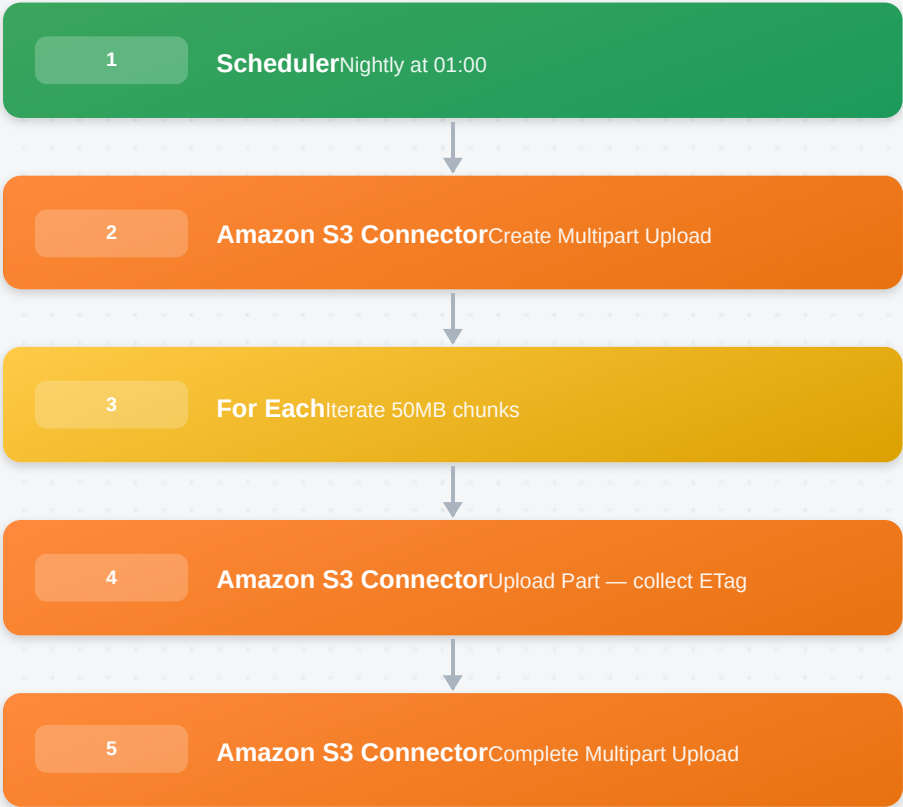
AWS S3 – Create/Upload Part/Complete Multipart Upload

CORE For Each

STEPS TO CREATE THE MULE FLOW

- 1 Call Create Multipart Upload to obtain an upload ID for the object key.
- 2 Split the export file into fixed-size chunks and upload each as a numbered part.
- 3 Collect each part's returned ETag as parts are uploaded.
- 4 Call Complete Multipart Upload with the full list of part numbers and ETags to assemble the object.

MULTIPART-EXPORT-UPLOAD-FLOW



MULE CONFIGURATION XML


```

<flow name="multipart-export-upload-flow">
  <scheduler doc:name="Nightly at 01:00">
    <scheduling-strategy>
      <cron expression="0 0 1 * * ?"/>
    </scheduling-strategy>
  </scheduler>
  <s3:create-multipart-upload config-ref="S3_Config" doc:name="Start
multipart upload"
    bucketName="exports-bucket" key="nightly-export.csv"
target="uploadId"/>
  <set-variable variableName="parts" value="#[]" />
  <foreach collection="#[readChunks('/exports/nightly-export.csv',
50 * 1024 * 1024)]">
    <s3:upload-part config-ref="S3_Config" doc:name="Upload chunk"
      bucketName="exports-bucket" key="nightly-export.csv"
      uploadId="#[vars.uploadId]" partNumber="#[vars.counter]"
target="partResult">
      <s3:content>#[payload]</s3:content>
    </s3:upload-part>
    <set-variable variableName="parts" value="#[vars.parts ++ [{
partNumber: vars.counter, eTag: vars.partResult.eTag }]]" />
  </foreach>
  <s3:complete-multipart-upload config-ref="S3_Config"
doc:name="Assemble object"
    bucketName="exports-bucket" key="nightly-export.csv"
    uploadId="#[vars.uploadId]" parts="#[vars.parts]" />
</flow>

```

EXPLANATION OF THE CODE

vars.parts accumulating { partNumber, eTag } per iteration

Complete Multipart Upload needs the exact list of part numbers and their ETags to verify every part arrived intact and in order, so the For

Each loop builds that list incrementally as each part upload confirms.

```
readChunks(..., 50 * 1024 * 1024)
```

Streaming the source file in fixed 50 MB chunks — well above S3's 5 MB minimum part size — keeps the total part count reasonable for a 2 GB file (around 40 parts) without ever holding the full file in memory at once.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** onto the canvas and configure a **Cron** expression for 01:00 nightly.
- 2 Drag the S3 **Create Multipart Upload** operation and set **Target Variable** to **uploadId**.
- 3 Drag a **For Each** scope; inside it, drag the S3 **Upload Part** operation, binding **Upload Id** and **Part Number**.
- 4 After each part, drag a **Set Variable** to append that part's number and ETag to the running **parts** list.
- 5 After the loop, drag the S3 **Complete Multipart Upload** operation and bind **Parts** to **vars.parts**, then save and run.

TESTING IN POSTMAN

This flow runs on a schedule rather than an HTTP trigger. For manual testing, expose a companion admin endpoint that invokes the same flow logic on demand.

POST

`https://localhost:8081/admin/export/run-now`

SAMPLE RESPONSE

JSON

```
{
  "bucket": "exports-bucket",
  "key": "nightly-export.csv",
  "partsUploaded": 41,
  "status": "COMPLETED"
}
```

BRIEFING ALL THE STEPS IN WORDS

A Create Multipart Upload call obtains an upload ID, a For Each loop uploads the large export file in 50 MB parts while tracking each part's ETag, and a final Complete Multipart Upload call assembles those parts into one object on S3's side — so a single dropped connection only costs one part's worth of retry, not the entire multi-gigabyte transfer.

#9

Moving a File from an Incoming Folder to an Archive Folder After Processing

AWS

PROBLEM STATEMENT

After a file has been successfully processed, it needs to move from the `incoming/` prefix to an `archive/` prefix in the same bucket — not stay forever mixed in with files still waiting to be handled.

DETAILED EXPLANATION

S3 has no native "move" or "rename" operation, because there's no real directory structure underneath — a move is always implemented as a copy to the new key followed by a delete of the old one. Doing this

correctly means the copy has to succeed before the delete runs, never the reverse, so a failed copy never results in a file vanishing entirely; this ordering is what makes the two-step "move" pattern safe to use anywhere a real move operation would otherwise be expected.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

AWS S3 Connector – Copy Object, Delete Object

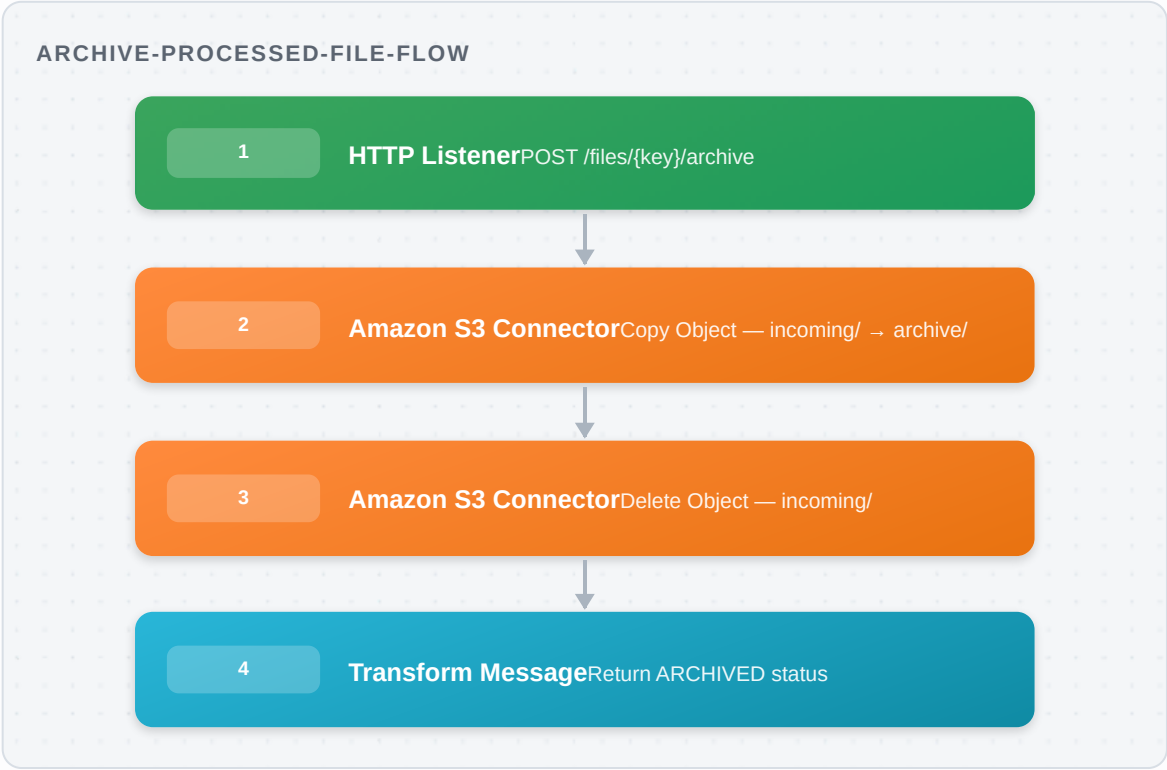
STEPS TO CREATE THE MULE FLOW

- 1

Expose an HTTP POST endpoint that names the file to archive.
- 2

Copy the object from the incoming/ prefix to the archive/ prefix in the same bucket.
- 3

Only after the copy succeeds, delete the original object from incoming/.



MULE CONFIGURATION XML

```

<flow name="archive-processed-file-flow">
  <http:listener path="/files/{key}/archive" config-
ref="HTTP_Listener_config"/>
  <set-variable variableName="archiveKey" value="#"
[attributes.uriParams.key replace 'incoming/' with 'archive/']"/>
  <s3:copy-object config-ref="S3_Config" doc:name="Copy to archive
prefix"
    sourceBucketName="documents-bucket" sourceKey="#"
[attributes.uriParams.key]"
    destinationBucketName="documents-bucket" destinationKey="#"
[vars.archiveKey]"/>
  <s3:delete-object config-ref="S3_Config" doc:name="Delete from
incoming"
    bucketName="documents-bucket" key="#"
[attributes.uriParams.key]"/>
  <ee:transform doc:name="Build response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{ archivedKey: vars.archiveKey, status: 'ARCHIVED' }]</ee:set-
payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

Copy Object placed before Delete Object in the flow

Mule executes flow components strictly in sequence, so if the copy throws an error, the flow's default error handling stops it right there — the delete component is never reached, and the original file stays exactly where it was.

`archiveKey` built by replacing `'incoming/'` with `'archive/'`

Deriving the destination key from the source key by a single targeted replace keeps the filename and any sub-folder structure underneath `incoming/` intact in the archive, rather than flattening everything into one folder.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/files/{key}/archive`.
- 2 Drag a `Set Variable` named `archiveKey` to compute the destination prefix.
- 3 Drag the S3 `Copy Object` operation first, then the S3 `Delete Object` operation directly after it — order matters here.
- 4 Drag a `Transform Message` to confirm the archive, then save and run.

TESTING IN POSTMAN

POST

`https://localhost:8081/files/incoming%2Finvoice-4471.pdf/archive`

SAMPLE RESPONSE

JSON

```
{
  "archivedKey": "archive/invoice-4471.pdf",
  "status": "ARCHIVED"
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts the key of a processed file, a Copy Object call duplicates it under the archive/ prefix, and only once that copy has succeeded does a Delete Object call remove the original

from incoming/ — simulating a move safely, in an order that never leaves the file lost if either step fails.

#10 Triggering a Mule Flow When a New File Lands in an S3 Bucket

AWS

PROBLEM STATEMENT

Files are dropped into an S3 bucket by an upstream batch job on no fixed schedule, and each new file needs to be picked up and processed automatically — without any change to the system doing the dropping.

DETAILED EXPLANATION

S3 has no push-based connector operation in Mule the way Salesforce has Streaming API — the standard, production-proven pattern instead is a Scheduler that periodically lists the bucket and compares what it finds against what's already been processed. Rather than re-listing the entire bucket's history every run, an Object Store is used as a watermark: it remembers which keys have already triggered processing, so each poll only acts on genuinely new ones. This keeps the pattern cheap even against a bucket with years of accumulated objects.

CONNECTORS & COMPONENTS USED

CORE Scheduler

AWS Amazon S3 Connector – List Objects

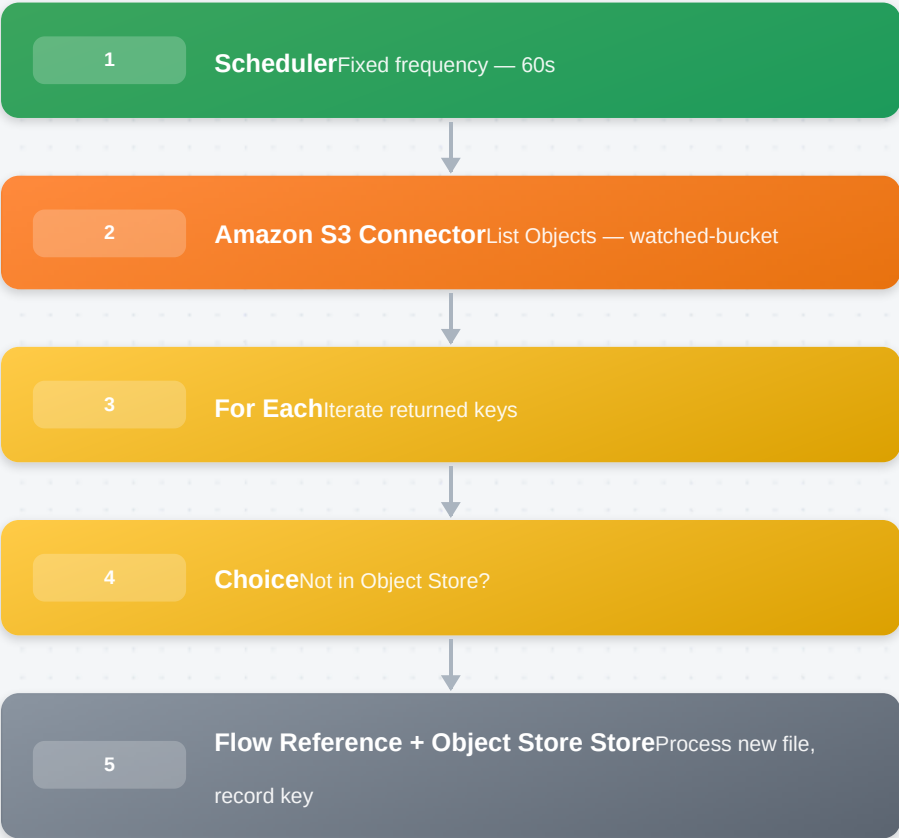
CORE Object Store

CORE For Each

STEPS TO CREATE THE MULE FLOW

- 1 Run a Scheduler every minute that lists objects under the watched prefix.
- 2 Filter out any key already present in the Object Store watermark.
- 3 For each genuinely new key, trigger downstream processing and record the key in the Object Store.

WATCH-NEW-FILES-FLOW



MULE CONFIGURATION XML


```

<flow name="watch-new-files-flow">
  <scheduler doc:name="Every 60 seconds">
    <scheduling-strategy>
      <fixed-frequency frequency="60000"/>
    </scheduling-strategy>
  </scheduler>
  <s3:list-objects config-ref="S3_Config" doc:name="List watched
bucket"
  bucketName="watched-bucket" prefix="incoming/">
    <foreach collection="#[payload]">
      <os:contains key="#[payload.key]"
objectStore="processedFilesStore" target="alreadySeen"/>
      <choice>
        <when expression="#[!vars.alreadySeen]">
          <flow-ref name="process-new-file-subflow"/>
          <os:store key="#[payload.key]"
objectStore="processedFilesStore">
            <os:value>#[now()]/</os:value>
          </os:store>
        </when>
      </choice>
    </foreach>
  </flow>

```

EXPLANATION OF THE CODE

os:contains checked before any processing happens

Checking the Object Store first, before doing any real work, is what stops the same file from being reprocessed on every single poll after it's already been handled once.

os:store called only inside the "not already seen" branch

The watermark is only written after processing has been kicked off, not before the check, so a

genuinely new key is guaranteed to be picked up exactly once per poll cycle rather than possibly being marked seen without ever being processed.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** onto the canvas and set **Fixed Frequency** to **60000** ms.
- 2 Drag the S3 **List Objects** operation and set **Bucket Name** and **Prefix**.
- 3 Drag a **For Each** scope, then inside it an **Object Store - Contains** operation bound to **processedFilesStore**.
- 4 Drag a **Choice** router checking **!vars.alreadySeen**, and inside it a **Flow Reference** to your processing subflow followed by **Object Store - Store**.
- 5 In the Mule Palette, create the **processedFilesStore** Object Store configuration with an appropriate expiration, then save and run.

TESTING IN POSTMAN

This flow is Scheduler-triggered, not HTTP-triggered. For on-demand testing, expose a companion admin endpoint that runs one poll cycle immediately.

POST

<https://localhost:8081/admin/watch-new-files/run-now>

SAMPLE RESPONSE

JSON

```
{
  "scanned": 14,
  "newFilesProcessed": 2,
  "skippedAlreadySeen": 12
}
```

BRIEFING ALL THE STEPS IN WORDS

A Scheduler polls the watched bucket every minute via List Objects, a For Each loop checks each returned key against an Object Store watermark, and only genuinely new keys trigger downstream processing before being recorded — simulating an event-driven S3 trigger with a poll-and-remember pattern that stays cheap and accurate no matter how large the bucket grows.

SECTION 2 Validation, Batch Processing & Governance (Problems 11–20)

#11 Validating a CSV File's Structure Before Uploading It to S3

AWS

PROBLEM STATEMENT

A partner uploads daily product-catalog CSV files, but a malformed file — missing a required column, or with a mismatched column count — needs to be rejected before it ever reaches S3, not discovered days later by whatever job reads it.

DETAILED EXPLANATION

Once a bad file lands in S3, it typically sits there until some downstream batch job trips over it, and by then the failure is far removed from its actual cause — hard to trace back to "this specific upload was malformed." Validating structure at the point of upload, before the S3 call ever happens, turns a delayed and confusing downstream failure into an immediate, specific rejection the partner can act on right away.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

DW Transform Message – CSV validation

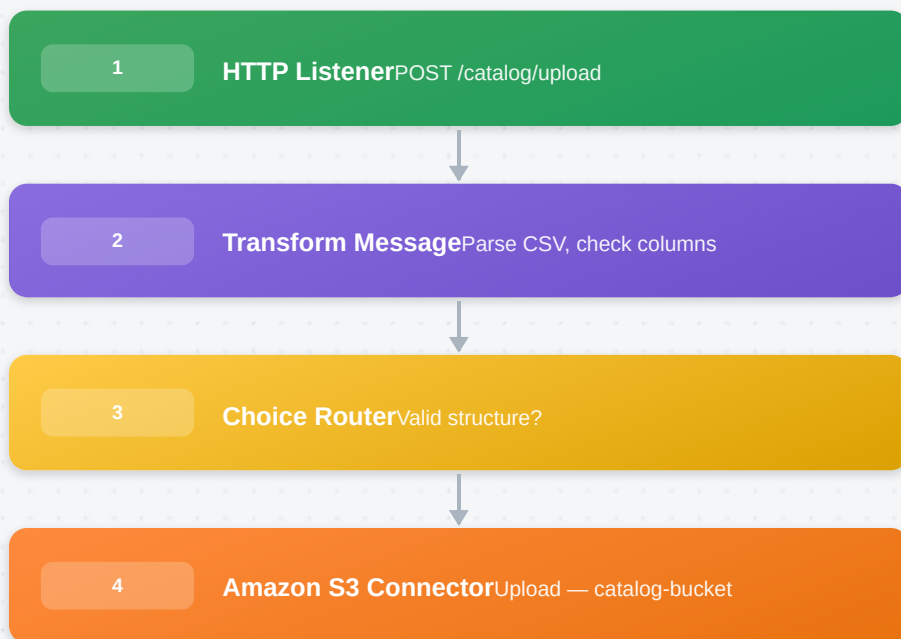
CORE Validation Module

AWS Amazon S3 Connector – Upload

STEPS TO CREATE THE MULE FLOW

- 1 Accept the CSV upload over HTTP and parse it with DataWeave's CSV reader.
- 2 Check that every row has the expected set of columns and that required columns are non-empty.
- 3 Reject the request with a detailed error if validation fails; otherwise upload the original file to S3.

VALIDATE-AND-UPLOAD-CATALOG-FLOW



MULE CONFIGURATION XML

```

<flow name="validate-and-upload-catalog-flow">
  <http:listener path="/catalog/upload" config-
ref="HTTP_Listener_config"/>
  <ee:transform doc:name="Validate CSV structure">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var rows = read(payload.parts.file.content, "application/csv")
var required = ["sku", "name", "price"]
---
{
  valid: (rows[0] default {}) pluck $$$ == required,
  rowCount: sizeof(rows)
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <choice>
    <when expression="#[payload.valid]">
      <s3:upload config-ref="S3_Config" doc:name="Upload catalog
file"
        bucketName="catalog-bucket" key="#['catalog/' ++ now() as
String {format: 'yyyyMMdd-HH:mm:ss'} ++ '.csv'}">
        <s3:content>#[payload.parts.file.content]</s3:content>
      </s3:upload>
    </when>
    <otherwise>
      <raise-error type="APP:INVALID_CSV_STRUCTURE"
description="Catalog CSV is missing required columns"/>
    </otherwise>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

```
read(payload.parts.file.content, "application/csv")
```

Parsing the raw upload as CSV
before doing anything else turns

the validation problem into a structural one DataWeave can reason about — column names and row shapes — rather than pattern-matching on raw text.

```
raise-error type="APP:INVALID_CSV_STRUCTURE"
```

Raising a custom, named error rather than just returning an error JSON body means this failure can be caught, logged, and handled distinctly anywhere else in the application that references this flow.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/catalog/upload`.
- 2 Drag a **Transform Message** and write the DataWeave script that parses the CSV and checks its header row.
- 3 Drag a **Choice** router checking `payload.valid`.
- 4 In the **when** branch, drag the S3 **Upload** operation; in **otherwise**, drag a **Raise Error** component with a custom error type.
- 5 Save and run, then test with a well-formed CSV and one missing a required column.

TESTING IN POSTMAN

POST

`https://localhost:8081/catalog/upload`

Header	Value
Content-Type	multipart/form-data

SAMPLE RESPONSE (INVALID FILE)

JSON

```
{
  "error": "APP:INVALID_CSV_STRUCTURE",
  "description": "Catalog CSV is missing required columns"
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts the CSV upload, a DataWeave transform parses it and checks its header row against the required column list, a Choice router only proceeds to the S3 Upload operation when the structure is valid, and a malformed file is rejected immediately with a specific, named error instead of silently landing in the bucket.

#12

Converting an Incoming JSON Payload to CSV and Storing It in S3

AWS

PROBLEM STATEMENT

An order-management system posts daily order data as JSON, but the finance team's reconciliation tool only reads CSV files dropped into a shared S3 bucket.

DETAILED EXPLANATION

This is one of the most common jobs a MuleSoft-AWS integration does: format translation combined with landing the result somewhere a legacy or third-party tool already expects to find it. DataWeave handles the JSON-to-CSV conversion declaratively in a single transform — no manual string concatenation of rows — and the S3 Upload operation lands the result exactly where the reconciliation tool's own scheduled job will pick it up.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

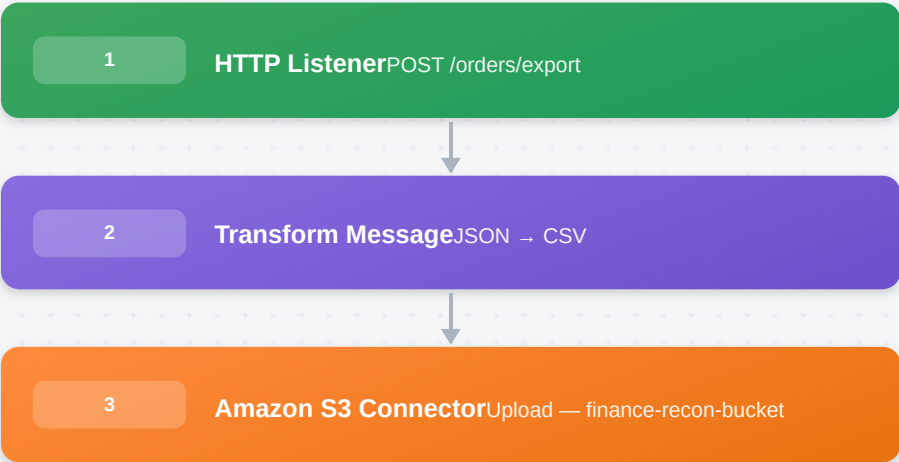
DW Transform Message – JSON to CSV

AWS Amazon S3 Connector – Upload

STEPS TO CREATE THE MULE FLOW

- 1 Accept the day's orders as a JSON array over HTTP.
- 2 Transform the array to CSV output using DataWeave.
- 3 Upload the resulting CSV bytes to the finance reconciliation bucket.

ORDERS-TO-CSV-FLOW



MULE CONFIGURATION XML


```

<flow name="orders-to-csv-flow">
  <http:listener path="/orders/export" config-
ref="HTTP_Listener_config"/>
  <ee:transform doc:name="JSON to CSV">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/csv
---
payload map {
  orderId: $.id,
  customer: $.customerName,
  total: $.totalAmount,
  orderDate: $.createdAt
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <s3:upload config-ref="S3_Config" doc:name="Upload orders CSV"
    bucketName="finance-recon-bucket"
    key="#['orders/' ++ now() as String {format: 'yyyy-MM-dd'} ++
'-orders.csv']">
    <s3:content>#[payload]</s3:content>
  </s3:upload>
</flow>

```

EXPLANATION OF THE CODE

output application/csv on the Transform Message

Setting the output MIME type is the entire conversion — DataWeave's CSV writer automatically handles header generation, delimiter escaping, and line endings from the mapped object structure.

key ending in '-orders.csv' using the current date

Naming the file by date rather than overwriting a fixed filename each day preserves a full history

in the bucket, which matters the first time finance needs to re-reconcile a prior day.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to **/orders/export**.
- 2 Drag a **Transform Message**, set the output to **application/csv**, and write the field mapping.
- 3 Drag the S3 **Upload** operation, set **Bucket Name** to **finance-recon-bucket**, and bind **Content** to **payload**.
- 4 Save and run, then verify the object in the S3 console opens correctly as CSV.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/export

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY



JSON

```
[
  { "id": "ORD-991", "customerName": "Maria Chen", "totalAmount":
214.50, "createdAt": "2026-08-28" }
]
```

SAMPLE RESPONSE



JSON

```
{ "bucket": "finance-recon-bucket", "key": "orders/2026-08-28-
orders.csv" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a JSON array of orders, a single DataWeave transform re-outputs it as CSV by changing only the output MIME type and field mapping, and the S3 Connector's Upload operation lands the result in a date-named key inside the finance reconciliation bucket — a clean handoff between two systems that will never speak each other's native format.

#13 Reading a CSV File from S3 and Processing Each Row in Batch

AWS

PROBLEM STATEMENT

A supplier drops a product-price-update CSV into S3 nightly, and every row needs to be validated and applied individually — with a report of which rows succeeded and which failed, not an all-or-nothing outcome.

DETAILED EXPLANATION

Processing a file row by row instead of as one giant operation is what makes partial failure survivable: one malformed row shouldn't block the other 4,999 good ones from going through. Mule's Batch Job component is purpose-built for exactly this — it loads the records, processes each independently with its own error isolation, and produces a final report of successes and failures, which is what lets an operations team act on exactly the rows that need attention instead of re-running the whole file.

CONNECTORS & COMPONENTS USED

AWS

Amazon S3 Connector – Get Object

CORE

Batch Job

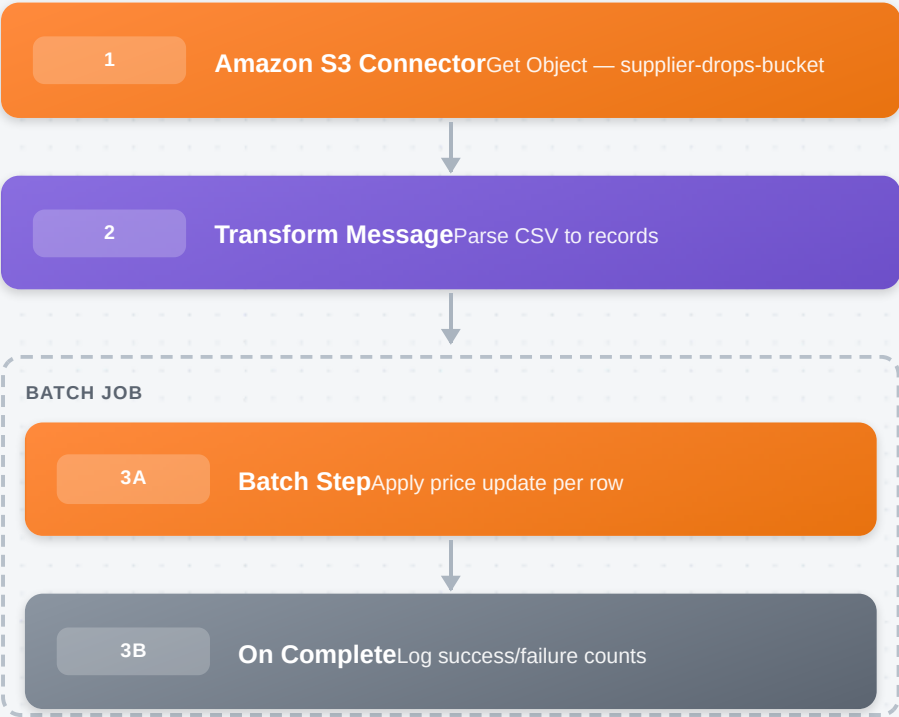
DW

Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Retrieve the CSV file from S3 and parse it into records.
- 2 Load each parsed row into a Batch Job for independent processing.
- 3 Apply each price update, isolating any row-level failure.
- 4 Summarize the batch result once every row has been attempted.

PROCESS-PRICE-UPDATE-CSV-FLOW



MULE CONFIGURATION XML

```

<flow name="process-price-update-csv-flow">
  <s3:get-object config-ref="S3_Config" doc:name="Get price update
file"
    bucketName="supplier-drops-bucket" key="prices/latest-
prices.csv"/>
  <ee:transform doc:name="Parse CSV rows">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/java
---
read(payload, "application/csv")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <batch:job jobName="price-update-batch">
    <batch:process-records>
      <batch:step name="applyPriceStep">
        <choice>
          <when expression="#[payload.sku != null and
payload.newPrice > 0]">
            <logger level="INFO" message="#[ 'Updated price for ' ++
payload.sku]"/>
          </when>
          <otherwise>
            <raise-error type="APP:INVALID_PRICE_ROW"
description="Missing SKU or invalid price"/>
          </otherwise>
        </choice>
      </batch:step>
    </batch:process-records>
    <batch:on-complete>
      <logger level="INFO"
        message="#[ 'Processed: ' ++ payload.successfulRecords as
String ++ ', Failed: ' ++ payload.failedRecords as String]"/>
    </batch:on-complete>
  </batch:job>
</flow>

```

```
</batch:job>  
</flow>
```

EXPLANATION OF THE CODE

`read(payload, "application/csv")` producing a Java array of records

Parsing to `application/java` hands the Batch Job component an in-memory collection it can split into individually-tracked records, which is the input shape `batch:job` expects.

`raise-error` inside the otherwise branch of the batch step

Raising an error for one record inside a batch step only fails that record — the batch framework catches it, marks it failed in the final report, and continues processing every other record in the file.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the S3 `Get Object` operation onto the canvas and set `Bucket Name` and `Key`.
- 2 Drag a `Transform Message` to parse the CSV bytes into a Java array of records.
- 3 Drag a `Batch Job` scope; inside `Process Records`, add a `Batch Step` with a `Choice` router validating each row.
- 4 In the `On Complete` phase, drag a `Logger` that reports `successfulRecords` and `failedRecords`.
- 5 Save and run, then check the console log for the final success/failure summary.

TESTING IN POSTMAN

This flow is triggered by file arrival, not HTTP. Expose an admin endpoint to run it on demand for testing.

POST

`https://localhost:8081/admin/price-update/run-now`

SAMPLE RESPONSE

JSON

```
{
  "totalRecords": 5000,
  "successfulRecords": 4988,
  "failedRecords": 12
}
```

BRIEFING ALL THE STEPS IN WORDS

The S3 Connector's Get Object operation retrieves the supplier's CSV, a transform parses it into individual records, and a Batch Job processes each row's price update independently — isolating any malformed row as a tracked failure rather than letting it stop the other several thousand rows from going through, then reporting the final tally once every row has been attempted.

#14

Checking an S3 Bucket's Versioning Status from a Mule Flow

AWS

PROBLEM STATEMENT

Before an automated cleanup job permanently deletes old objects from a bucket, it needs to confirm the bucket has versioning enabled — because a versioned bucket recovers from an accidental delete, and an unversioned one doesn't.

DETAILED EXPLANATION

Bucket versioning is configured at the bucket level, once, usually outside of any Mule flow — but a flow that's about to run a destructive operation shouldn't simply assume that configuration is in place. Querying the actual versioning status first, and refusing to proceed with the destructive step if it isn't enabled, turns an assumption that could silently be wrong into an explicit, checked precondition.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

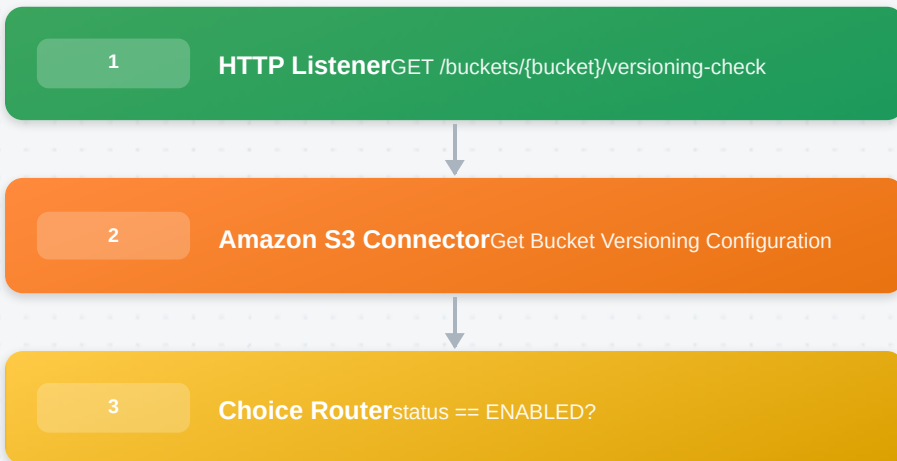
AWS S3 Connector – Get Bucket Versioning Configuration

CORE Validation Module

STEPS TO CREATE THE MULE FLOW

- 1 Expose an admin endpoint that takes a bucket name and confirms it's safe to run cleanup.
- 2 Call Get Bucket Versioning Configuration for that bucket.
- 3 Reject the request if the status isn't ENABLED, rather than silently proceeding.

CHECK-VERSIONING-BEFORE-CLEANUP-FLOW



MULE CONFIGURATION XML


```

<flow name="check-versioning-before-cleanup-flow">
  <http:listener path="/buckets/{bucket}/versioning-check" config-
ref="HTTP_Listener_config"/>
  <s3:get-bucket-versioning-configuration config-ref="S3_Config"
doc:name="Get versioning status"
  bucketName="#[attributes.uriParams.bucket]"
target="versioningStatus"/>
  <choice>
    <when expression="#[vars.versioningStatus.status == 'ENABLED']">
      <set-payload value="#[{ safeToCleanup: true }]" />
    </when>
    <otherwise>
      <raise-error type="APP:VERSIONING_NOT_ENABLED"
description="Bucket is not versioned; cleanup refused" />
    </otherwise>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

target="versioningStatus" holding the connector's result

Storing the check result in a variable rather than the payload keeps it available to the Choice router's expression without needing a separate read of the original payload afterward.

raise-error rather than a soft warning in the otherwise branch

Making the unsafe case a hard error, not a logged warning that's easy to miss, is what actually stops a cleanup job from running against an

unversioned
bucket by mistake.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/buckets/{bucket}/versioning-check`.
- 2 Drag the S3 **Get Bucket Versioning Configuration** operation, bind **Bucket Name** to the URI parameter, and set **Target Variable**.
- 3 Drag a **Choice** router checking the status, with a **Raise Error** in the otherwise branch.
- 4 Save and run, then test against a versioned and an unversioned bucket.

TESTING IN POSTMAN

GET

`https://localhost:8081/buckets/documents-bucket/versioning-check`

SAMPLE RESPONSE

JSON

```
{ "safeToCleanup": true }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a bucket name, the S3 Connector's Get Bucket Versioning Configuration operation confirms whether versioning is actually turned on, and a Choice router only allows a destructive cleanup job to proceed when it is — converting a dangerous assumption into a checked, enforced precondition.

#15

Applying an S3 Lifecycle Rule Check Before Archiving Old Files

AWS

PROBLEM STATEMENT

Before an application-level "archive old files" job runs, it needs to confirm the target bucket's S3 Lifecycle rule for transitioning objects to cheaper storage is actually configured — so the app-level archival logic and the bucket's own storage-class transition don't work against each other.

DETAILED EXPLANATION

S3 Lifecycle rules can automatically transition objects to Glacier or delete them entirely, entirely independent of anything a Mule flow does. If an application also has its own notion of "old" files that it archives manually, the two mechanisms need to agree, or a file could be moved by the app while a lifecycle rule is also mid-transition on it. Reading the bucket's lifecycle configuration first lets the flow confirm both mechanisms are aligned — or refuse to run if they aren't — before touching anything.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

AWS S3 Connector – Get Bucket Lifecycle Configuration

CORE Choice Router

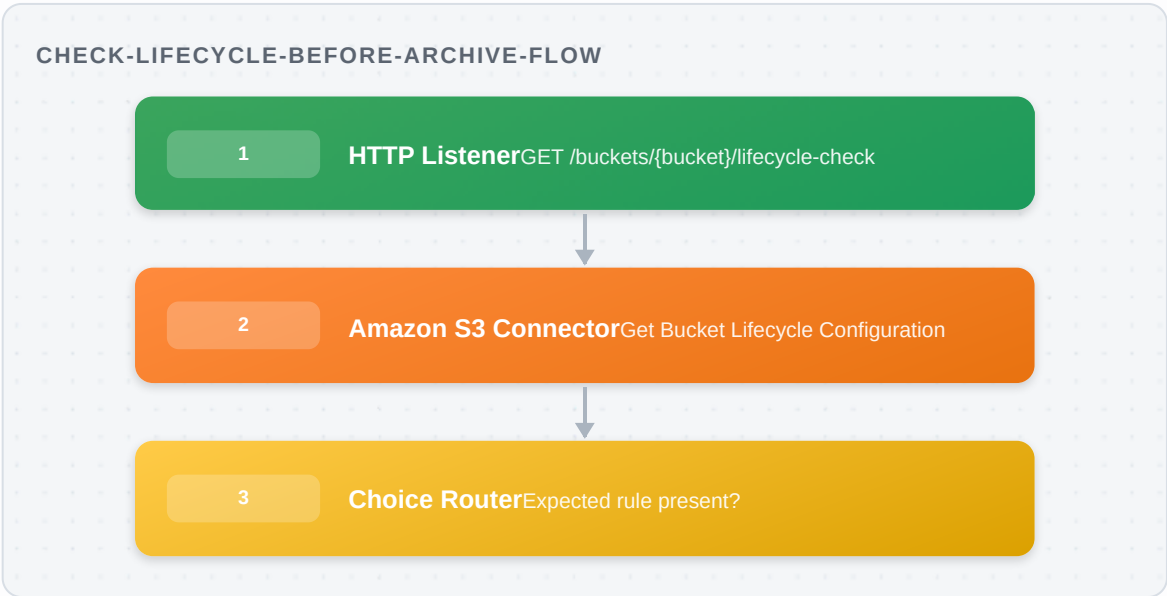
STEPS TO CREATE THE MULE FLOW

- 1

Expose an admin endpoint that names the bucket to check before archival.
- 2

Call Get Bucket Lifecycle Configuration for that bucket.
- 3

Confirm an expected rule ID exists before allowing the app-level archive job to run.



MULE CONFIGURATION XML

```

<flow name="check-lifecycle-before-archive-flow">
  <http:listener path="/buckets/{bucket}/lifecycle-check" config-
ref="HTTP_Listener_config"/>
  <s3:get-bucket-lifecycle-configuration config-ref="S3_Config"
doc:name="Get lifecycle rules"
  bucketName="#[attributes.uriParams.bucket]"
target="lifecycleRules"/>
  <choice>
    <when expression="#[vars.lifecycleRules.rules some ($.id ==
'archive-to-glacier-after-90-days')]">
      <set-payload value="#[{ safeToArchive: true }]" />
    </when>
    <otherwise>
      <raise-error type="APP:LIFECYCLE_RULE_MISSING"
description="Expected archival lifecycle rule not found on bucket"/>
    </otherwise>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

`vars.lifecycleRules.rules some ($.id == '...')`

DataWeave's `some` operator checks whether any rule in the returned list matches the expected rule ID, which is more resilient than assuming rules always come back in a fixed order or position.

`a hardcoded rule ID rather than a generic "any rule exists" check`

Checking for the specific rule this application depends on, not just any lifecycle configuration at all, is what

actually confirms the two systems agree — a bucket could have lifecycle rules configured for something else entirely.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/buckets/{bucket}/lifecycle-check`.
- 2 Drag the S3 **Get Bucket Lifecycle Configuration** operation and set **Target Variable** to `lifecycleRules`.
- 3 Drag a **Choice** router checking for the expected rule ID using DataWeave's **some** operator.
- 4 Save and run against a bucket with the rule configured and one without it.

TESTING IN POSTMAN

GET

`https://localhost:8081/buckets/documents-bucket/lifecycle-check`

SAMPLE RESPONSE

JSON

```
{ "safeToArchive": true }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a bucket name, the S3 Connector's Get Bucket Lifecycle Configuration operation retrieves the bucket's actual transition rules, and a Choice router confirms the specific rule this application depends on is present before letting an app-level archival job proceed — keeping the

bucket's own automatic storage-class transitions and the application's manual archival logic from working at cross purposes.

#16

Encrypting an S3 Object Using Server-Side Encryption on Upload

AWS

PROBLEM STATEMENT

Customer PII documents uploaded through a Mule API must be encrypted at rest in S3 as a compliance requirement, verifiable by anyone auditing the bucket's contents.

DETAILED EXPLANATION

S3's server-side encryption (SSE-S3) encrypts every object with an AES-256 key that AWS manages entirely — it costs nothing extra, adds no client-side complexity, and requires only a single attribute on the upload request. For most compliance requirements that specify "encrypted at rest" without further detail, SSE-S3 is the simplest way to satisfy the requirement, and setting it explicitly on every upload from Mule, rather than relying on a bucket-level default that could later be changed, makes the encryption guarantee visible right in the integration code.

CONNECTORS & COMPONENTS USED

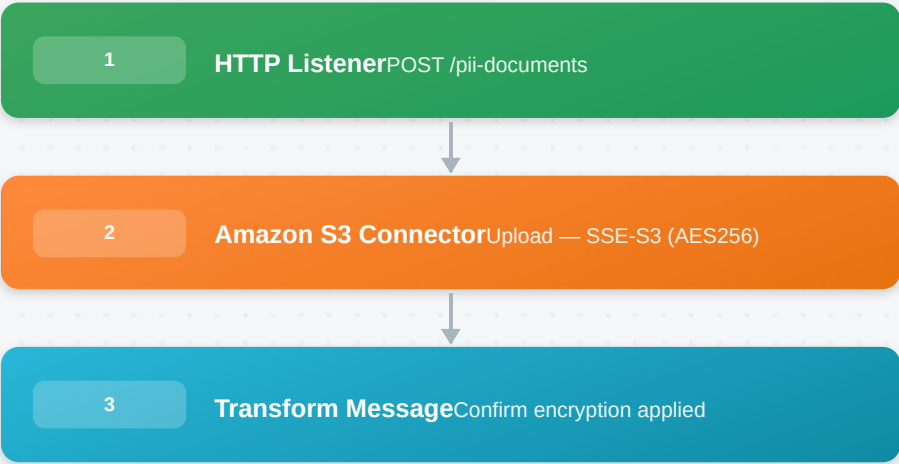
CORE HTTP Listener

AWS Amazon S3 Connector – Upload (SSE-S3)

STEPS TO CREATE THE MULE FLOW

- 1 Expose an HTTP POST endpoint for PII document uploads.
- 2 Call the S3 Upload operation with server-side encryption explicitly set to AES256.
- 3 Confirm the encryption setting was applied by reading it back from the upload response.

UPLOAD-ENCRYPTED-PII-FLOW



MULE CONFIGURATION XML

```

<flow name="upload-encrypted-pii-flow">
  <http:listener path="/pii-documents" config-
ref="HTTP_Listener_config"/>
  <s3:upload config-ref="S3_Config" doc:name="Upload with SSE-S3"
    bucketName="pii-documents-bucket"
    key="#['pii/' ++ uuid() ++ '.pdf']"
    serverSideEncryption="AES256" target="uploadResult">
    <s3:content>#[payload.parts.file.content]</s3:content>
  </s3:upload>
  <ee:transform doc:name="Confirm encryption">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  encrypted: vars.uploadResult.serverSideEncryption == 'AES256',
  algorithm: vars.uploadResult.serverSideEncryption
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

`serverSideEncryption="AES256"` set explicitly on the upload call

Setting encryption on the request itself, rather than depending on a bucket-level default encryption setting, means the requirement is enforced by the integration code

and shows up in code review — it can't be silently disabled by a bucket configuration change nobody on the integration team notices.

key using `uuid()` rather than the original filename

Generating a random key for PII documents avoids leaking any information (a customer's name, an account number) that the original filename might have carried into the S3 key itself, which is otherwise visible to anyone with list permissions on the bucket.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/pii-documents`.
- 2 Drag the S3 `Upload` operation, set `Bucket Name`, and in the operation's advanced settings set `Server Side Encryption` to `AES256`.
- 3 Set `Target Variable` to `uploadResult` so the response can confirm the setting.
- 4 Drag a `Transform Message` to build the confirmation response, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/pii-documents

Header

Value

Content-Type

multipart/form-data

SAMPLE RESPONSE



JSON

```
{ "encrypted": true, "algorithm": "AES256" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a PII document upload, the S3 Connector's Upload operation stores it under a randomly generated key with server-side AES-256 encryption explicitly requested, and a transform confirms the encryption setting was actually applied — making a compliance requirement an enforced, visible part of the integration rather than an assumption about bucket defaults.

#17

Restricting S3 Uploads by File Type and Size Using a Mule Validation Flow

AWS

PROBLEM STATEMENT

A public-facing document upload API needs to reject anything that isn't a PDF or PNG, and anything over 10 MB, before it ever reaches S3 — not after storage costs and abuse risk have already been incurred.

DETAILED EXPLANATION

Any endpoint that accepts arbitrary file uploads from the public internet is a target for abuse: oversized files that drive up storage costs, or disguised executables uploaded as if they were documents. Validating both the declared content type and the actual byte size in Mule, before the S3 Upload operation ever runs, keeps that abuse surface entirely out of the storage layer — S3 never even sees a rejected file.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

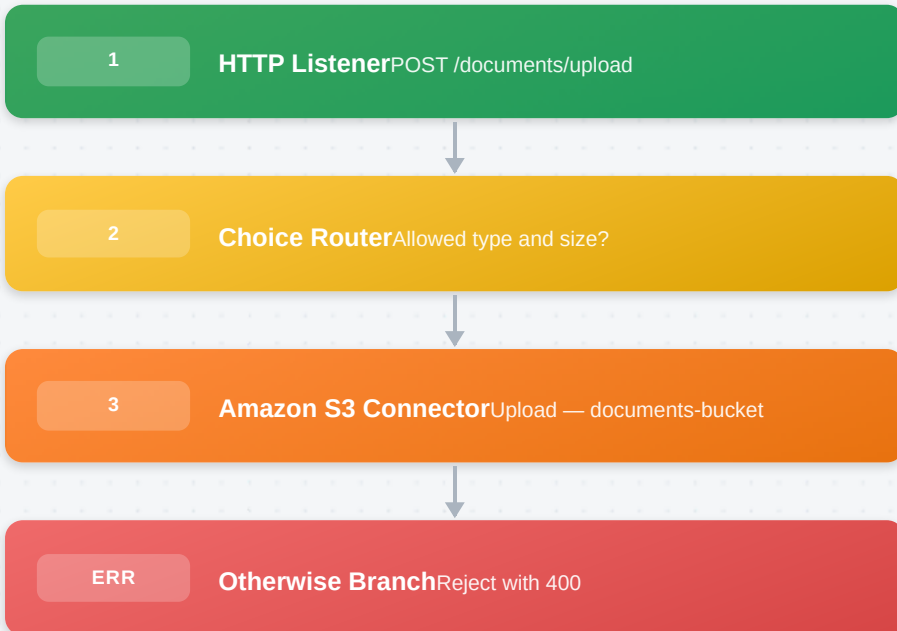
CORE Choice Router

AWS Amazon S3 Connector – Upload

STEPS TO CREATE THE MULE FLOW

- 1 Accept the upload and read its content type and byte size from the multipart metadata.
- 2 Reject anything not in the allowed content-type list, or over the size limit.
- 3 Only call S3 Upload once both checks pass.

RESTRICTED-UPLOAD-FLOW



MULE CONFIGURATION XML

```

<flow name="restricted-upload-flow">
  <http:listener path="/documents/upload" config-
ref="HTTP_Listener_config"/>
  <choice>
    <when expression="#['application/pdf','image/png'] contains
payload.parts.file.^mimeType
    and sizeof(payload.parts.file.content) <= 10485760]">
      <s3:upload config-ref="S3_Config" doc:name="Upload document"
        bucketName="documents-bucket"
        key="#['documents/' ++ uuid() ++ '-' ++
payload.parts.file.filename]">
        <s3:content>#[payload.parts.file.content]</s3:content>
      </s3:upload>
    </when>
    <otherwise>
      <raise-error type="APP:UPLOAD_REJECTED" description="File type
not allowed or exceeds 10MB limit"/>
    </otherwise>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

```
['application/pdf','image/png'] contains payload.parts.file.^mimeType
```

Checking against an explicit allow-list, rather than a deny-list of disallowed types, defaults to rejecting anything unexpected

— the safer posture for a public endpoint, since a deny-list has to anticipate every bad type in advance.

```
sizeof(payload.parts.file.content) <= 10485760
```

Checking the actual byte size of the received content, not a client-supplied size header that could be wrong or missing, is what makes the size check trustworthy against a caller that doesn't cooperate.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to **/documents/upload**.
- 2 Drag a **Choice** router and write the combined type-and-size expression in its **when** branch.
- 3 Inside **when**, drag the S3 **Upload** operation; inside **otherwise**, drag a **Raise Error** component.
- 4 Save and run, then test with an allowed small PDF, a disallowed file type, and an oversized file.

TESTING IN POSTMAN

POST

https://localhost:8081/documents/upload

Header

Value

Content-Type

multipart/form-data

SAMPLE RESPONSE (REJECTED FILE)

JSON

```
{
  "error": "APP:UPLOAD_REJECTED",
  "description": "File type not allowed or exceeds 10MB limit"
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts the upload, a Choice router checks the actual received content type against an allow-list and the actual byte size against a 10 MB cap in a single expression, and only a file that passes both checks ever reaches the S3 Upload operation — keeping abuse and storage-cost risk out of the bucket entirely rather than cleaning it up afterward.

#18

Syncing Files Between an On-Prem SFTP Server and an S3 Bucket

AWS

PROBLEM STATEMENT

A legacy on-prem system drops files onto an SFTP server every hour, and those files need to end up in S3 within minutes so cloud-based downstream systems can consume them, without decommissioning the SFTP server itself.

DETAILED EXPLANATION

This is one of the most common hybrid-integration problems: a legacy system that can't be changed continues writing to the interface it's always used, while a MuleSoft flow bridges that interface to the cloud storage modern systems expect. The SFTP connector polls for new files the same way it always

would in an on-prem-to-on-prem integration; the only difference here is that instead of writing the result to another filesystem, the flow streams it straight into S3.

CONNECTORS & COMPONENTS USED

CORE

SFTP Connector – On New or Updated File

AWS

Amazon S3 Connector – Upload

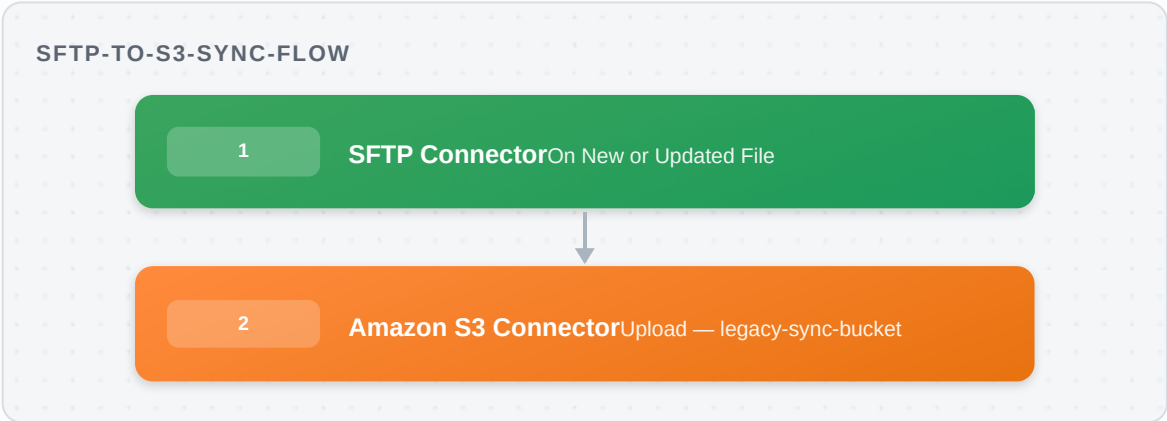
STEPS TO CREATE THE MULE FLOW

- 1

Configure the SFTP connector's file-arrival trigger to watch the drop directory.
- 2

Stream each new file's content directly into the S3 Upload operation.
- 3

Let the SFTP connector's built-in post-processing action move or delete the source file once uploaded.



MULE CONFIGURATION XML

```

<flow name="sftp-to-s3-sync-flow">
  <sftp:listener config-ref="SFTP_Config"
    directory="/outbound/legacy-drops"
    doc:name="Watch legacy drop directory">
    <sftp:matcher filenamePattern="*.dat"/>
    <sftp:post-processing-action>
      <sftp:move-to-directory directory="/outbound/processed"
        renameTo="#[attributes.fileName]"/>
    </sftp:post-processing-action>
  </sftp:listener>
  <s3:upload config-ref="S3_Config" doc:name="Upload to S3"
    bucketName="legacy-sync-bucket"
    key="#['legacy/' ++ attributes.fileName]">
    <s3:content>#[payload]</s3:content>
  </s3:upload>
</flow>

```

EXPLANATION OF THE CODE

sftp:post-processing-action moving the file after the listener fires

Configuring the SFTP listener to move the file to a processed directory once its own polling cycle completes prevents the same file from being picked up and uploaded to S3 a second time

on the next
poll.

s3:content bound directly to payload

The SFTP listener delivers the file's content as the flow's payload, so it streams straight into the S3 Upload operation without an intermediate transform — the file's bytes never need reshaping, only relocating.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **SFTP - On New or Updated File** source onto the canvas and configure the **SFTP_Config** with the legacy server's host and credentials.
- 2 Set **Directory** to **/outbound/legacy-drops** and **Filename Pattern** to ***.dat**.
- 3 Under **Post Processing Action**, choose **Move to Directory** and set it to **/outbound/processed**.
- 4 Drag the S3 **Upload** operation after the listener and set **Bucket Name** and **Key**, then save and run.

TESTING IN POSTMAN

This flow triggers on SFTP file arrival, not HTTP. Verify by placing a test file on the SFTP server and checking S3 directly, or expose an admin status endpoint.

GET <https://localhost:8081/admin/sftp-sync/status>

SAMPLE RESPONSE

JSON

```
{
  "lastSyncedFile": "legacy/export-20260828-0300.dat",
  "syncedAt": "2026-08-28T03:00:14Z"
}
```

BRIEFING ALL THE STEPS IN WORDS

The SFTP connector's file-arrival listener watches the legacy drop directory and delivers each new file's content directly into the flow, the S3 Connector's Upload operation streams it straight into the

cloud bucket, and the SFTP listener's own post-processing action moves the source file aside so it's never picked up twice — a bridge that lets an unmodified legacy system feed modern cloud consumers.

#19 Tagging S3 Objects Dynamically Based on Payload Content

AWS

PROBLEM STATEMENT

Uploaded documents need to be tagged with their department and sensitivity level so a separate governance process can apply the right retention and access policy — without that governance process having to inspect file contents itself.

DETAILED EXPLANATION

S3 object tags are key-value pairs attached to an object, separate from its actual content and from user-defined metadata, and specifically designed to be queried and acted on by other AWS services and policies — lifecycle rules, access policies, and cost-allocation reports can all key off tags. Setting tags at upload time, derived from whatever the flow already knows about the document (its department, its classification), means governance tooling downstream never needs to open the file to know how to treat it.

CONNECTORS & COMPONENTS USED

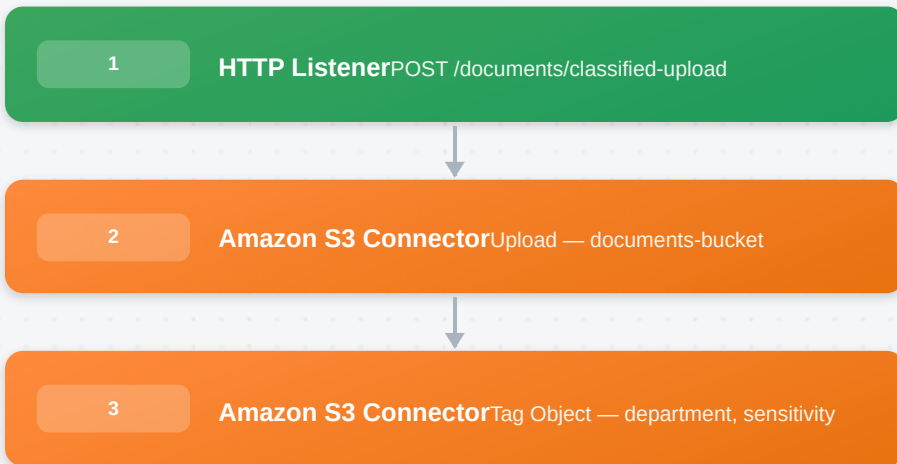
CORE HTTP Listener

AWS Amazon S3 Connector – Upload, Tag Object

STEPS TO CREATE THE MULE FLOW

- 1 Accept the upload along with its department and sensitivity classification.
- 2 Upload the file to S3.
- 3 Call Tag Object immediately after, applying the classification as object tags.

UPLOAD-AND-TAG-DOCUMENT-FLOW



MULE CONFIGURATION XML

```
<flow name="upload-and-tag-document-flow">
  <http:listener path="/documents/classified-upload" config-
ref="HTTP_Listener_config"/>
  <set-variable variableName="docKey" value="#['documents/' ++
uuid() ++ '-' ++ payload.parts.file.filename]"/>
  <s3:upload config-ref="S3_Config" doc:name="Upload document"
  bucketName="documents-bucket" key="#[vars.docKey]"/>
  <s3:content>#[payload.parts.file.content]</s3:content>
</s3:upload>
  <s3:tag-object config-ref="S3_Config" doc:name="Apply
classification tags"
  bucketName="documents-bucket" key="#[vars.docKey]"/>
  <s3:tags>#{[
  department: payload.parts.department.content as String,
  sensitivity: payload.parts.sensitivity.content as String
]}</s3:tags>
  </s3:tag-object>
</flow>
```

EXPLANATION OF THE CODE

`vars.docKey` computed once, reused in both S3 calls

Both the Upload and the Tag Object calls need to reference the exact same key, so computing it once into a variable guarantees the tags land on precisely the object that was just uploaded, not a recalculated (and potentially different) key.

Tag Object called as a separate step after Upload

The connector's Upload operation doesn't accept tags directly in this configuration, so tagging happens as its own explicit call immediately afterward — sequenced so the object always exists before its tags are set.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/documents/classified-upload`.
- 2 Drag a **Set Variable** named `docKey` to compute the object key once.
- 3 Drag the S3 **Upload** operation, binding **Key** to `vars.docKey`.
- 4 Drag the S3 **Tag Object** operation, bind the same **Key**, and set the **Tags** map from the request's department and sensitivity fields.
- 5 Save and run, then verify the tags in the S3 console's Properties tab for the uploaded object.

TESTING IN POSTMAN

POST

https://localhost:8081/documents/classified-upload

Header

Value

Content-Type

multipart/form-data

SAMPLE REQUEST (POSTMAN BODY → FORM-DATA)

FORM-DATA

```
file: contract-4471.pdf
department: legal
sensitivity: confidential
```

SAMPLE RESPONSE

JSON

```
{
  "key": "documents/8f2a1c-contract-4471.pdf",
  "tags": { "department": "legal", "sensitivity": "confidential" }
}
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a document along with its department and sensitivity classification, the S3 Connector's Upload operation stores the file, and a follow-up Tag Object call attaches that classification as native S3 object tags — giving a separate governance process everything it needs to apply retention and access policy without ever having to open the file itself.

#20

Building a File-Archival Flow That Moves Processed Files from a Landing Bucket to a Processed Bucket

AWS

PROBLEM STATEMENT

Closing out Volume 1: a nightly job needs to find every file in a "landing" bucket older than 24 hours that's been marked as processed, move each into a "processed" bucket organized by date, and report exactly what it moved.

DETAILED EXPLANATION

This problem combines almost everything the volume has covered: listing objects by prefix (#3), checking metadata before acting (#7), the copy-then-delete move pattern (#9), and batch-style per-item processing (#13) — assembled into the single, end-to-end archival job a real project actually schedules to run every night. Building it as one coherent flow, rather than as separate manually-triggered pieces, is what makes it something operations can trust to run unattended.

CONNECTORS & COMPONENTS USED

CORE Scheduler

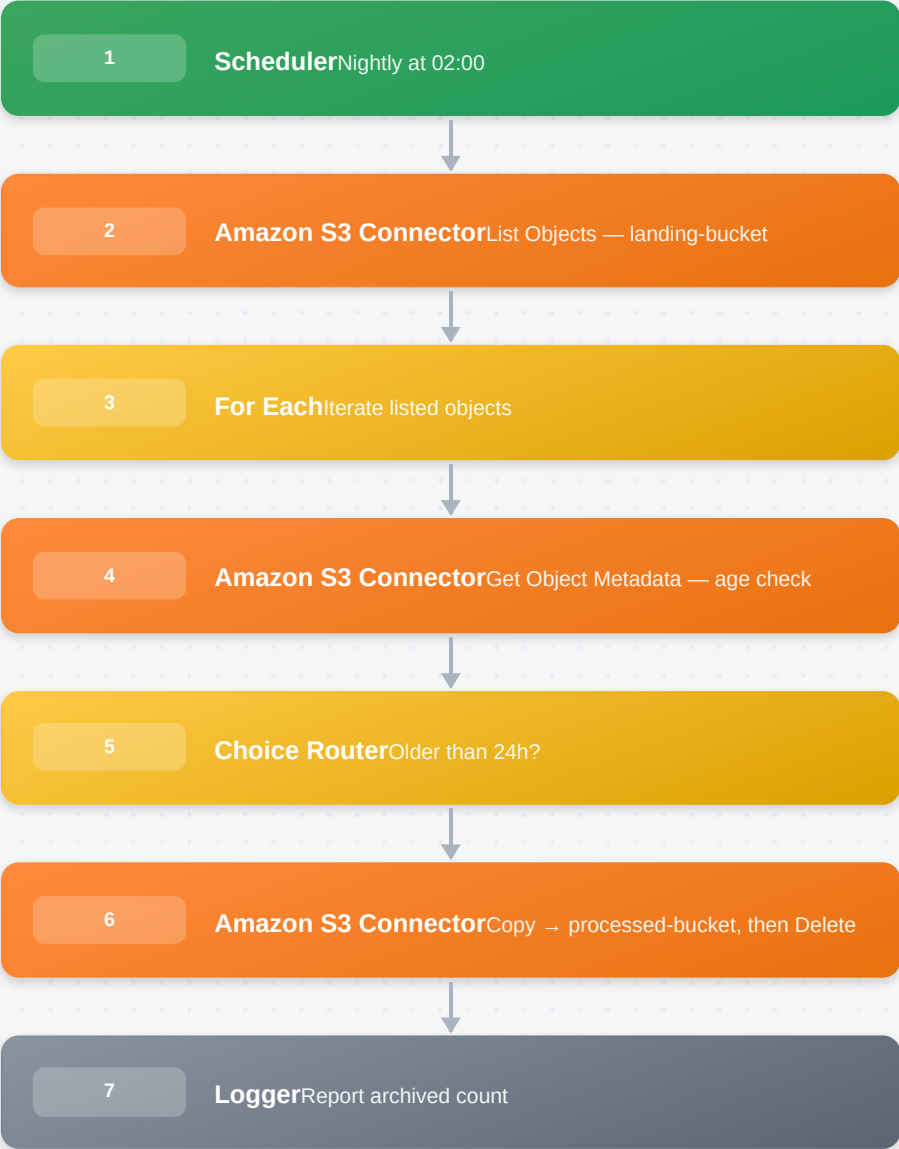
AWS S3 Connector – List, Get Object Metadata, Copy, Delete

CORE For Each

STEPS TO CREATE THE MULE FLOW

- 1 Run nightly and list every object in the landing bucket tagged as processed.
- 2 For each one, check its metadata to confirm it's older than 24 hours.
- 3 Copy it into the processed bucket under a date-partitioned key, then delete the landing copy.
- 4 Report the total count of files archived.

NIGHTLY-ARCHIVAL-FLOW



MULE CONFIGURATION XML

```

<flow name="nightly-archival-flow">
  <scheduler doc:name="Nightly at 02:00">
    <scheduling-strategy>
      <cron expression="0 0 2 * * ?"/>
    </scheduling-strategy>
  </scheduler>
  <s3:list-objects config-ref="S3_Config" doc:name="List landing
bucket"
  bucketName="landing-bucket" prefix="processed-tagged/">
    <set-variable variableName="archivedCount" value="#[0]"/>
    <foreach collection="#[payload]">
      <s3:get-object-metadata config-ref="S3_Config" doc:name="Check
file age"
      bucketName="landing-bucket" key="#[payload.key]"
target="fileMeta"/>
      <choice>
        <when expression="#[(now() - vars.fileMeta.lastModified) >
|PT24H|]">
          <s3:copy-object config-ref="S3_Config" doc:name="Copy to
processed bucket"
          sourceBucketName="landing-bucket" sourceKey="#
[payload.key]"
          destinationBucketName="processed-bucket"
          destinationKey="#[(now() as String {format:
'yyyy/MM/dd'}) ++ '/' ++ payload.key]"/>
          <s3:delete-object config-ref="S3_Config" doc:name="Delete
from landing"
          bucketName="landing-bucket" key="#[payload.key]"/>
          <set-variable variableName="archivedCount" value="#
[vars.archivedCount + 1]"/>
        </when>
      </choice>
    </foreach>
    <logger level="INFO" message="#['Archived ' ++ (vars.archivedCount

```



```
as String) ++ ' files']"/>
</flow>
```

EXPLANATION OF THE CODE

```
(now() - vars.fileMeta.lastModified) >> |PT24H|
```

DataWeave's Period comparison operator lets the age check read almost like plain language — comparing the elapsed time since last modification directly against a 24-hour ISO-8601 duration, rather than converting everything to raw milliseconds by hand.

```
Copy then Delete nested inside the same when branch as #9
```

Reusing the safe copy-before-delete ordering from Problem #9 inside this loop means a failure partway through one file's move never risks that specific file being lost — it simply stays in the landing bucket to be picked up again the following night.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** onto the canvas and configure a **Cron** expression for 02:00 nightly.
- 2 Drag the S3 **List Objects** operation scoped to the tagged-as-processed prefix.
- 3 Drag a **Set Variable** named **archivedCount** initialized to **0**, then a **For Each** scope.
- 4 Inside the loop, drag S3 **Get Object Metadata**, a **Choice** router with the age expression, then S3 **Copy Object** and **Delete Object** inside the **when** branch.
- 5 After each successful move, increment **archivedCount**; after the loop, drag a **Logger** to report the final total, then save and run.

TESTING IN POSTMAN

This flow is Scheduler-triggered. Expose an admin endpoint to run one archival pass on demand for testing.

POST

`https://localhost:8081/admin/archival/run-now`

SAMPLE RESPONSE

JSON

```
{
  "scanned": 37,
  "archivedCount": 29,
  "skippedTooRecent": 8
}
```

BRIEFING ALL THE STEPS IN WORDS

A nightly Scheduler lists every tagged-as-processed object in the landing bucket, a For Each loop checks each one's age via Get Object Metadata, and anything older than 24 hours is copied into a date-partitioned key in the processed bucket and then deleted from landing — closing Volume 1 by combining listing, metadata checks, the safe copy-then-delete move, and batch iteration into the single scheduled job a real project would actually run every night.

End of Volume 1

Volume 1 built the foundation every MuleSoft–AWS integration rests on: uploading, downloading, listing, copying, and securely archiving objects in Amazon S3, plus the practical file-handling patterns — presigned URLs, multipart uploads, metadata checks, tagging, and SFTP-to-S3 bridging — that a real project actually needs, not just the textbook basics.

Volume 2 moves from storing files to moving messages: Amazon SQS, SNS, and EventBridge, and the queueing, fan-out, and event-routing patterns that connect MuleSoft to the rest of an event-driven AWS architecture — Problems 21–40.